

An Internship Report on
Agent Orchestration Framework with LangChain

BY
Sidram Bajrang Patil

Under the Guidance
of
Ms. M. H. Khan



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on
“Agent Orchestration Framework with LangChain”

Submitted to
DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE

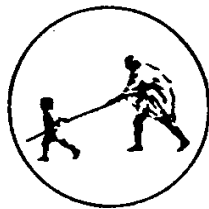
for fulfillment of the requirement for the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By
Sidram Bajrang Patil

Under the Guidance
of

Ms. M. H. Khan

(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)

Academic Year 2025-26

Certificate



This is to certify that the Internship entitled

“Agent-Orchestration Framework with LangChain”

*being submitted by **Mr. Sidram Bajrang Patil** to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.*

Ms. M. H. Khan

Project Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Engg., Nanded

INFOSYS VIRTUAL INTERNSHIP 6.0 OFFER LETTER



Batch 13 Set-2 | Project Onboarding | Infosys Springboard Virtual Internship 6.0

1 message

springboardmentor <springboardmentor556@gmail.com>
Bcc: s22_patil_sidram@mgmcen.ac.in

Mon, 16 Feb 2026 at 10:27 pm

Dear Interns,

Welcome to the Infosys Virtual Internship 6.0 program!

This mail is to Inform that you are Onboarded into Batch 13 Set-2 of Infosys Springboard Virtual Internship Program.

Project assigned to you is : Agent-Orchestration Framework with LangChain

Please do find the meeting details attached & join the classes regularly with the same credentials.

Project Name: Agent-Orchestration Framework with LangChain

Scheduled Class Timing: 8.30pm-9.30pm

Start Date: 17-Feb-2026

🔗 Class Meeting Link:

Please join the sessions regularly using this dedicated URL:

<https://meet.google.com/cky-hafr-ydj>

We look forward to seeing you at the first session!

Best Regards,
Devender Pratap

INFOSYS VIRTUAL INTERNSHIP 6.0 COMPLETION LETTER



Congratulations on successfully completing the Infosys Springboard Internship 6.0

1 message

Infosys Springboard-Support <Springboard-support@infosys.com>

Thu, 14 May 2026 at 10:07 am



Dear Intern,

Congratulations! You have successfully completed Infosys Springboard Internship 6.0! We're proud of your achievement and the dedication you've shown throughout the internship.

Your certificate will be sent to your registered email address shortly. Keep up the great work and continue striving for excellence!

Announce your achievement and your internship experience on social media. Tag us @InfosysSpringboard when you do.

Note: Your project is your asset. Refrain from sharing your artifacts in online forums/domains to safeguard your work's ownership.

The name on the certificate will be as per the profile name registered on the Infosys Springboard platform.

Regards,
Team Infosys Springboard

Keep your learning journey going. Scan the QR code to download the **Infosys Springboard App** and access your courses on the go!



App Store



Play Store

Follow us on Facebook | Instagram | LinkedIn | X (Twitter) | Threads

In case of queries, write to springboard-support@infosys.com.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Ms. M. H. Khan**, for her invaluable support, guidance, and encouragement throughout this project. Her profound knowledge and expertise have been instrumental in the successful completion of this work. Her patience and willingness to assist me at every step have greatly enriched my learning experience. Her constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). I am heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, I am also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

Sidram Bajrang Patil

[B.Tech CSE-A]

ABSTRACT

The Agent-Orchestration Framework with LangChain is an intelligent software system developed to improve the handling of user queries through coordinated agent-based processing. The framework combines large language models, semantic retrieval, memory management, and external tool integration within a unified architecture. Instead of relying on a single execution flow, the system analyses incoming requests and dynamically routes them to the most appropriate processing pipeline based on the query type and complexity.

The framework consists of multiple execution pipelines designed for general conversations, tool-assisted tasks, content generation, document understanding, and advanced reasoning. Specialized agents work together to perform query analysis, information retrieval, response generation, validation, and refinement. Document-based queries are processed using a retrieval mechanism that utilizes vector embeddings and semantic search techniques to provide context-aware answers. The system also maintains conversational history and persistent user information to support personalized and consistent interactions.

To enhance functionality, the framework integrates external services such as search, calculator, weather, and conversion utilities. The combination of intelligent routing, agent collaboration, retrieval mechanisms, and memory management enables the system to generate accurate and relevant responses across different application scenarios. The implementation demonstrates how orchestration-driven architectures can improve the flexibility, scalability, and effectiveness of modern AI-powered assistants while supporting complex real-world tasks.

Keywords: Agent Orchestration, LangChain, Multi-Agent System, Artificial Intelligence, Semantic Search, Retrieval-Augmented Generation, FAISS, Memory Management, Large Language Models, Intelligent Assistant.

TABLE OF CONTENTS

INTERNSHIP OFFER LETTER	I
INTERNSHIP COMPLETION LETTER	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
TABLE OF CONTENTS	V-VII
LIST OF FIGURES	VIII
LIST OF TABLES	IX
Chapter 1. INTRODUCTION	1
1.1 Internship Overview	2
1.2 Company Profile - Infosys & Infosys Springboard	3
1.3 Overview of the Project	4
1.4 Problem Statement	6
1.5 Need for agent Orchestration	7
1.6 Objectives of the Project	8
1.7 Scope of the Project	9
1.8 Internship Milestones and Learning Outcomes	10
1.9 Methodology Overview	12
Chapter 2. TECHNOLOGY STACK AND TRAINING	14
2.1 Python	14
2.2 Flask Web Framework	16
2.3 Conversational AI and Large Language Model	17
2.4 LangChain Framework	18
2.4.1 Prompt Templates	19
2.4.2 Chains	20
2.4.3 Agents	21
2.4.4 Memory Systems	22
2.5 RAG and Vector Databases	23
2.5.1 Document Embeddings	24

2.5.2 FAISS Vector Search	25
2.6 Multi-Agent Systems	26
2.7 Frontend Technologies	27
2.8 Database and Storage	28
2.9 Version Control and Deployment	29
Chapter 3. SYSTEM ANALYSIS AND DESIGN	31
3.1 Requirement Specification	31
3.1.1 Functional Requirements	31
3.1.2 Non- Functional Requirements	32
3.2 Query Processing Techniques	32
3.2.1 Input Analysis	33
3.2.2 Regex-Based Pre-Routing	34
3.2.3 LLM Router Classification	35
3.2.4 Output Formatting	36
3.3 Proposed System Architecture	36
3.4 Workflow Orchestration Pipelines	38
3.4.1 SIMPLE Pipeline	40
3.4.2 Tool Pipeline	41
3.4.3 CONTENT Pipeline	42
3.4.4 COMPLEX Pipeline	43
3.4.5 DOCUMENT Pipeline	43
3.5 Muti-Agent Architecture	45
3.5.1 Routing Agents	47
3.5.2 Research and Analysis Agents	48
3.5.3 Domain Experts Agents	49
3.5.4 Content Creation Agents	50
3.5.5 Quality Assurance Agents	50
3.6 Memory Management System	51
3.6.1 Conversation Buffer Memory	52
3.6.2 FAISS Vector Memory	53
3.6.3 Persistent User Memory	54

3.7 Database Design	55
Chapter 4. SYSTEM IMPLEMENTATION	57
4.1 Backend Implementation	57
4.2 API Endpoints and Session Management	58
4.3 Tool Integration Implementation	60
4.4 File Processing and Document Handling	63
4.5 Frontend Implementation	64
4.5.1 Landing Page Interface	65
4.5.2 User Registration and Login System	65
4.5.3 Main Chat Dashboard	66
4.5.4 File Upload and Document Interaction	67
4.6 Analytics Dashboard	67
4.7 Deployment and Execution Environment	68
Chapter 5. RESULTS AND EVALUATION	70
5.1 Functional Testing	70
5.2 Performance Evaluation	71
5.3 System Results	73
5.4 Advantages of the System	76
5.5 Limitations of the System	77
5.6 Future Enhancements	78
CONCLUSION	79
REFERENCES	80

List of Figures

Figure No.	Name of Figure	Page No.
1.1	Workflow of the Multi-Agent Orchestration Framework	5
1.2	Comparison Between Traditional Chatbot and Multi-Agent Orchestration Framework	8
2.1	Python	15
2.2	Flask	16
2.3	FAISS Persistent Memory Workflow	25
2.4	Git and GitHub	29
2.5	Hugging Face	30
3.1	Overall System Architecture	37
3.2	Pipeline Execution Workflow	39
3.3	RAG-Based Document Processing Workflow	44
3.4	Multi-Agent Architecture Diagram	46
3.5	Memory Management Workflow	51
3.6	Database Schema Diagram (ER Diagram)	55
4.1	Backend Request–Response Workflow	58
4.2	API Communication Workflow	59
4.3	Tool Execution Workflow	60
4.4	File Processing Pipeline	63
4.5	Landing Page Screenshot	65
4.6	Registration and Login Interface	66
4.7	Main Chat Dashboard Screenshot	66
4.8	File Upload and RAG Interaction	67
5.1	SIMPLE Pipeline Output	73
5.2	TOOL Pipeline Output	74
5.3	CONTENT Pipeline Output	74
5.4	COMPLEX Pipeline Output	75
5.5	DOC Pipeline Output	75

List of Tables

Table No.	Name of Table	Page No.
1.1	Internship Milestone Timeline	12
3.1	Complete Agent Registry	47
5.1	Functional Test Cases	71
5.2	Pipeline Response Time Analysis	72

INTRODUCTION

Artificial Intelligence has become an important technology in modern software development. Many applications now use intelligent systems to improve user interaction, automate tasks, process information, and generate responses efficiently. Traditional chatbot systems are capable of answering basic questions, but they often face difficulties while handling complex workflows, memory management, document understanding, and external tool execution[4].

To overcome these limitations, modern AI applications use agent-based architectures and workflow orchestration systems. In these systems, different agents perform specialized tasks such as routing, analysis, research, summarization, and response generation. This approach improves response quality, system organization, and contextual understanding.

During the virtual internship conducted by Infosys through the Infosys Springboard platform, practical knowledge was gained in Artificial Intelligence, backend development, workflow orchestration, and multi-agent systems. The internship provided exposure to modern technologies and real-world software development practices used in intelligent applications.

An intelligent agent orchestration framework was developed using Python, Flask, LangChain, Large Language Models (LLMs), and vector database technologies. The system was designed to process user requests through different execution pipelines and specialized agents instead of relying on a single chatbot workflow. The framework includes functionalities such as intelligent query routing, multi-agent communication, Retrieval-Augmented Generation (RAG), vector-based memory retrieval, document processing, external tool integration, and analytics management. Different tools were integrated into the system for performing tasks such as mathematical calculations, web searching, weather information retrieval, unit conversion, and document-based querying[4].

The project also implements a memory management system using FAISS vector storage for maintaining contextual information and retrieving relevant data from previous interactions and uploaded documents. This improves conversational continuity and enables the generation of more context-aware responses.

The internship experience provided practical understanding of:

- AI agent workflows,
- routing mechanisms,
- memory systems,
- tool integration,
- backend API development,
- database management,
- intelligent workflow execution.

The internship helped in understanding the design and implementation of intelligent orchestration systems capable of handling complex workflows and advanced user interactions using modern AI technologies.

1.1 Internship Overview

The virtual internship was completed through the Infosys Springboard platform organized by Infosys. The internship focused on the domain of Artificial Intelligence and intelligent workflow systems. During the internship, an agent orchestration framework using LangChain was developed as part of the technical learning and implementation process.

The internship provided practical understanding of modern AI technologies and workflow automation systems. Different learning modules and milestone-based tasks were completed during the development process. The training mainly focused on Python programming, LangChain framework, intelligent agents, tool integration, memory systems, and multi-agent orchestration workflows.

During the initial phase of the internship, understanding was gained about LangChain fundamentals including prompts, chains, agents, and conversational workflows. A basic conversational agent was created to understand how language models process user queries and generate responses. In the next phase, external tool integration was implemented using LangChain tools and APIs. Different tools such as calculator and weather APIs were connected with the agent workflow to perform specialized operations dynamically. Error handling and tool invocation logic were also studied during this phase.

Further learning involved multi-agent orchestration and memory management systems. Different agents were designed for specific tasks such as routing, research, analysis, and summarization. Memory mechanisms including conversation memory and vector-based retrieval were implemented to improve contextual understanding and

workflow continuity. The final phase focused on workflow automation, API integration, frontend interaction, and system optimization. Backend API development, frontend dashboard interaction, workflow execution, documentation, and testing were completed as part of the project implementation process.

The internship provided practical exposure to:

- intelligent agent systems,
- workflow orchestration,
- Retrieval-Augmented Generation (RAG),
- vector memory systems,
- backend API development,
- tool integration,
- frontend interaction mechanisms.

The internship helped in understanding how modern AI systems are designed using multi-agent communication, workflow routing, memory retrieval, and intelligent automation techniques.

1.2 Company Profile - Infosys & Infosys Springboard

Infosys is a global information technology and consulting company founded in 1981 and headquartered in Bengaluru, India. The company provides services in software development, cloud computing, Artificial Intelligence, cybersecurity, data analytics, and digital transformation. Infosys is recognized as one of the leading IT service providers and supports organizations across different industries through technology-based solutions.

Infosys Springboard is a digital learning platform developed by Infosys to provide technical education, certification programs, coding practice, and virtual internship opportunities for students and learners. The platform offers learning programs in different domains such as Python programming, Artificial Intelligence, Web Development, Data Science, Cloud Computing, and other modern technologies.

The internship and project development activities were completed through the Python domain learning pathway available on the Infosys Springboard platform. Through this domain, practical exposure was gained in Python programming, backend development, Artificial Intelligence concepts, workflow automation, and intelligent agent systems. As part of the internship activities, the project related to agent orchestration using LangChain was assigned. The vision of Infosys focuses on

delivering technology-driven innovation and digital transformation solutions. The organization also supports technical learning and skill development initiatives through platforms such as Infosys Springboard, which help students gain practical industry-level experience through project-based learning and virtual internships.

1.3 Overview of the Project

The developed system is a multi-agent AI orchestration platform designed to process different categories of user queries through intelligent routing, specialized agents, and workflow-based execution pipelines. The platform was developed using Python, Flask, HTML, CSS, JS, LangChain, and Groq LLaMA 3.1 models for implementing intelligent conversational workflows and automated task execution.

The system uses a multi-agent architecture in which different AI agents perform specialized operations such as routing, research, analysis, summarization, content generation, and response validation. Instead of processing all requests through a single workflow, the platform analyses the query type and routes it to the appropriate execution pipeline. This improves workflow organization, response quality, and contextual understanding.

The platform consists of:

- 16 specialized AI agents,
- 5 integrated live tools,
- 5 execution pipelines are SIMPLE, TOOL, CONTENT, COMPLEX, and DOC.

Each pipeline is designed for handling a specific category of operations. The SIMPLE pipeline processes direct conversational queries, while the TOOL pipeline handles external tool execution such as calculations and web-based operations. The CONTENT pipeline is used for structured content generation tasks, and the COMPLEX pipeline manages multi-step reasoning and advanced workflow execution. The DOC pipeline processes uploaded documents and performs document-based querying using Retrieval-Augmented Generation (RAG) techniques.

The framework also integrates multiple live tools for performing operations such as:

- mathematical calculations,
- web searching,
- weather information retrieval,
- unit conversion,
- system time processing.

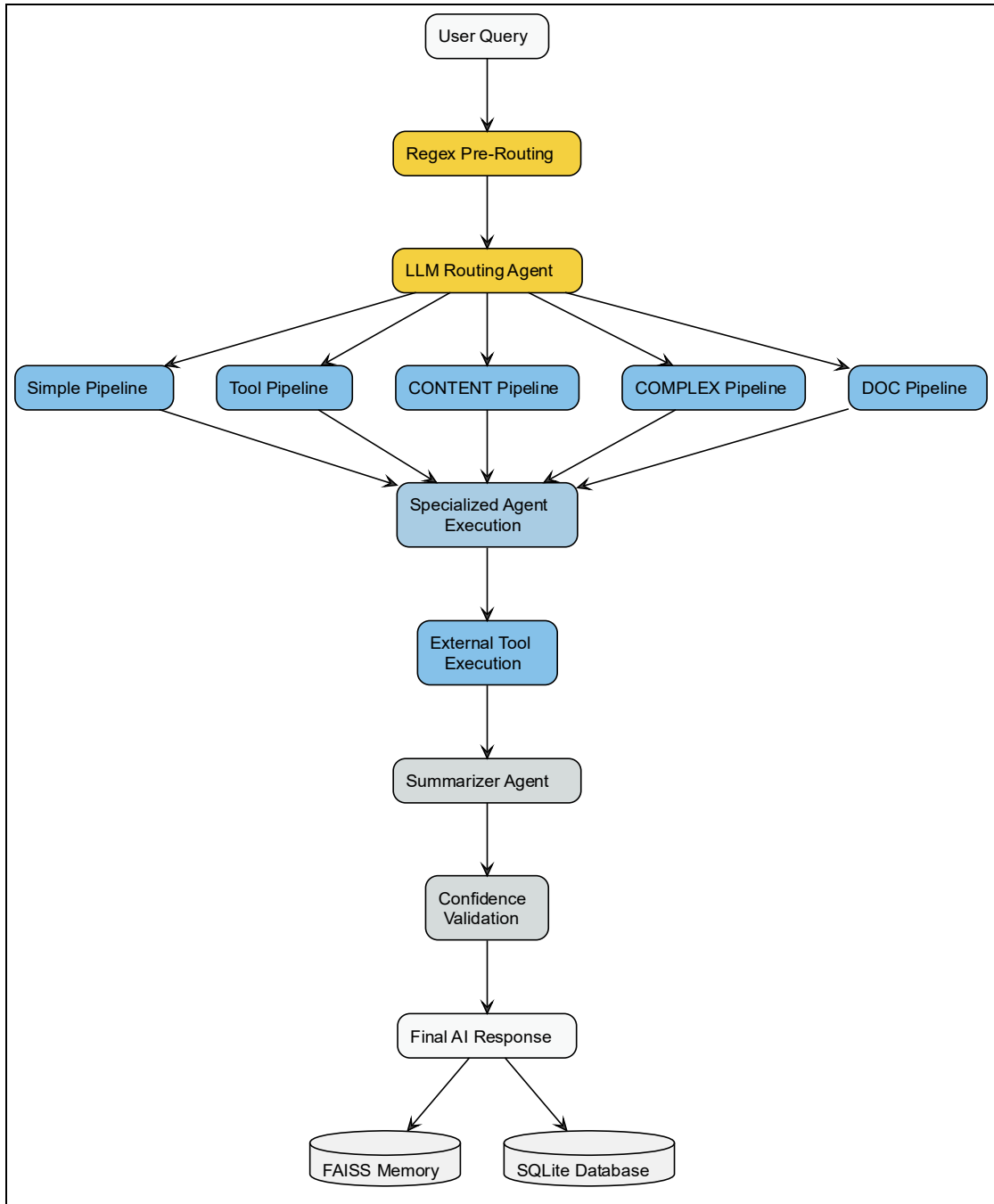


Fig 1.1: Workflow of the Multi-Agent Orchestration Framework

Document understanding functionality was implemented using Retrieval-Augmented Generation (RAG) and vector-based retrieval mechanisms. Uploaded PDF, DOCX, CSV, and presentation files are processed using embeddings and similarity search techniques to retrieve relevant contextual information before generating responses. FAISS vector storage was used for implementing memory retrieval and document search functionality. For deployment and accessibility, the application was containerized using Docker and deployed on Hugging Face Spaces. This allowed the

system to run as a web-accessible AI platform capable of handling workflow orchestration and multi-agent execution in an online environment.

The platform also includes features such as:

- intelligent query routing,
- persistent memory handling,
- chat history management,
- analytics dashboard,
- workspace management,
- quick prompt execution,
- pin message functionality.

Overall, the project demonstrates the implementation of an intelligent orchestration framework capable of combining multi-agent communication, workflow automation, external tool integration, document retrieval, and memory management within a single AI-powered platform.

1.4 Problem Statement

Traditional chatbot systems are mainly designed to generate responses using a single conversational workflow. Although these systems can answer basic user queries, they face several limitations while handling complex operations, contextual understanding, and workflow execution.

One major limitation of traditional chatbots is the absence of long-term memory management. Most systems cannot retain information across different sessions, which results in loss of conversational context and repeated user interactions. This problem reduces personalization and affects response continuity [4].

Single-agent conversational systems also face difficulties while processing different categories of queries efficiently. A single workflow is often unable to manage tasks such as research, analysis, document retrieval, content generation, and tool execution simultaneously. This creates performance and scalability challenges in intelligent applications.

Another limitation is the lack of real-time tool integration. Many chatbot systems cannot directly access external utilities such as:

weather information, mathematical calculations, web search, unit conversion, current time, system-based operations. As a result, these systems provide limited functionality for real-world problem solving. Traditional systems also do not support efficient

document understanding and contextual retrieval. Uploaded files such as PDF, DOCX, and CSV documents cannot be processed intelligently without Retrieval-Augmented Generation (RAG) and vector-based search mechanisms. This restricts the ability of the system to generate context-aware responses from document content [5].

In addition, many conversational systems do not include workflow monitoring, analytics management, or performance tracking mechanisms. Without these functionalities, it becomes difficult to evaluate system performance, monitor execution pipelines, analyse response accuracy, and improve workflow efficiency.

To overcome these limitations, a multi-agent orchestration framework was developed using intelligent routing, workflow pipelines, external tool integration, memory retrieval systems, and document-based contextual processing.

1.5 Need for Agent Orchestration

Modern AI applications are required to perform multiple tasks such as reasoning, research, document analysis, content generation, tool execution, and memory handling. A single-agent system often faces difficulties in managing all these operations efficiently within one workflow. This affects response quality, scalability, and overall system performance. Agent orchestration improves the system by dividing responsibilities among multiple specialized agents. Each agent is designed to perform a specific task such as routing, research, analysis, summarization, or tool execution. This specialization increases workflow efficiency and helps generate more accurate and context-aware responses.

Multi-agent systems also improve scalability because new agents and tools can be added without affecting the entire architecture. Different workflows can operate independently, making the system easier to maintain. This approach also improves task management for complex user requests involving multiple operations.

A multi-agent orchestration framework can be compared to a hospital environment where different specialists handle different medical problems. Instead of depending on one general doctor for every treatment, patients are referred to specialized experts such as surgeons, cardiologists, or neurologists based on the problem type. Similarly, in an orchestration system, different agents handle different categories. Therefore, agent orchestration provides a more organized, scalable, and intelligent approach for developing advanced AI systems capable of handling complex workflows and diverse user interactions [4].

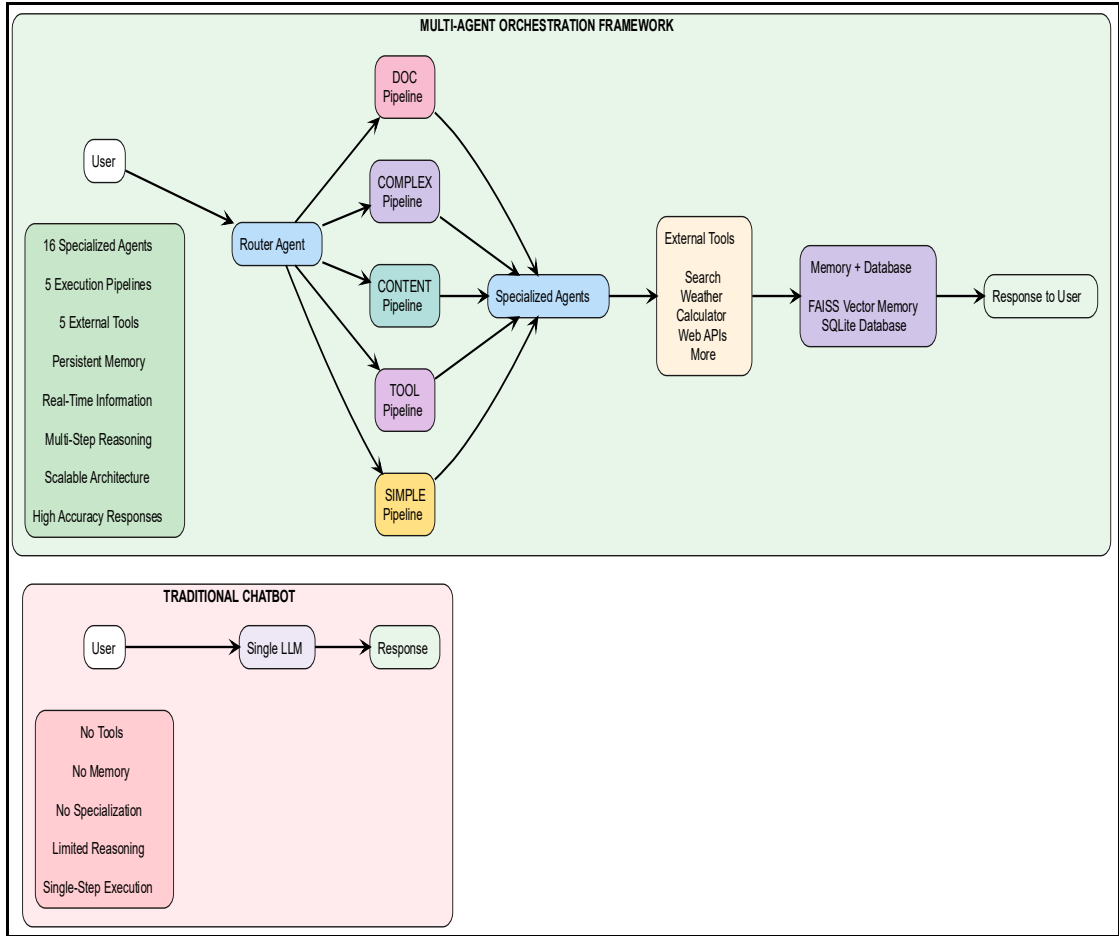


Fig 1.2: Comparison Between Traditional Chatbot and Multi-Agent Orchestration Framework

1.6 Objectives of the Project

The main objective of the project is to develop an intelligent multi-agent orchestration system capable of handling different categories of user queries through specialized workflows, integrated tools, memory systems, and document retrieval mechanisms. The system is designed to improve conversational intelligence, workflow management, and response quality compared to traditional chatbot architectures.

One of the primary objectives of the project is to build a multi-agent orchestration framework with intelligent routing across five execution pipelines named SIMPLE, TOOL, CONTENT, COMPLEX, and DOC. Each pipeline is responsible for handling specific categories of operations based on the user query type. The routing mechanism analyses incoming requests and forwards them to the most suitable workflow for efficient execution and response generation. Another important objective is to integrate real-time external tools into the AI workflow. The system includes five integrated tools for performing operations such as mathematical calculations, weather

information retrieval, web searching, unit conversion, and system time processing. These tools help the platform perform real-world operations dynamically instead of relying only on static conversational responses.

The project also aims to implement Retrieval-Augmented Generation (RAG) for document-based querying and contextual information retrieval. The framework supports processing of uploaded PDF, DOCX, CSV, XLSX, and PPTX files. Uploaded documents are converted into vector embeddings and processed using similarity search techniques to retrieve relevant contextual information before generating responses. Another major objective is to develop a persistent neural core memory system using the FAISS vector database. This memory mechanism helps the system maintain contextual continuity by storing and retrieving relevant information from previous conversations and uploaded documents. The memory system improves personalization, contextual understanding, and response consistency during user interactions.

The project also focuses on developing a full-featured dashboard for managing workflows and user interactions. The dashboard includes functionalities such as analytics monitoring, chat history management, search functionality, workspace organization, and message management features. These functionalities help in monitoring system activity and improving user experience.

The final objective of the project is to deploy the application using Docker on Hugging Face Spaces for online accessibility and workflow execution. Containerized deployment improves portability, scalability, and environment consistency, allowing the system to operate as a web-accessible AI orchestration platform.

1.7 Scope of the Project

The scope of the project is centred on developing an intelligent multi-agent orchestration platform capable of managing different categories of user requests through workflow-based execution, specialized AI agents, memory systems, and external tool integration. The system is designed to improve conversational intelligence, task automation, and contextual response generation using modern Artificial Intelligence technologies.

The platform supports intelligent query routing through multiple execution pipelines for handling conversational queries, tool execution, document processing, content generation, and complex reasoning tasks. Different workflows are implemented to improve response accuracy, workflow organization, and execution efficiency.

The system integrates real-time external tools for operations such as mathematical calculations, weather information retrieval, web searching, unit conversion, and system time processing. These functionalities extend the capabilities of the platform beyond traditional chatbot systems.

The project also supports document understanding using Retrieval-Augmented Generation (RAG) and vector-based retrieval mechanisms. Uploaded files such as PDF, DOCX, CSV, XLSX, and PPTX documents can be processed to retrieve relevant contextual information and generate document-aware responses [5].

Another important scope of the project is the implementation of persistent memory management using the FAISS vector database [6]. The memory system helps maintain conversational continuity and improves contextual understanding by storing and retrieving relevant information from previous interactions and uploaded documents. The platform includes dashboard functionalities such as analytics monitoring, workspace management, history handling, search operations, quick prompts, and message management features to improve workflow monitoring and user interaction.

The modular architecture of the system also provides future expansion capability for integrating advanced workflow automation services such as travel booking systems, appointment scheduling, ticket reservation platforms, email automation, and enterprise-level task management systems through external APIs and intelligent workflow execution mechanisms. The project can further be extended using cloud deployment, parallel agent execution, advanced reasoning models, voice-based interaction systems, and real-time enterprise automation workflows for large-scale intelligent applications.

1.8 Internship Milestones and Learning Outcomes

The internship was organized through milestone-based implementation and evaluation activities focused on LangChain, intelligent agents, workflow orchestration, API integration, memory systems, and AI-based automation. Each milestone introduced new concepts and practical tasks that gradually transformed the system from a basic conversational agent into a complete multi-agent orchestration framework.

Milestone 1: Environment Setup and Basic Agent Creation

The first milestone focused on setting up the Python and LangChain development environment and understanding the core building blocks of LangChain including

LLMs, prompts, chains, and agents. A basic conversational agent was developed to process static user queries using prompt-based interaction. Prompt templates, agent types, and console-based interaction workflows were also implemented during this phase.

Milestone 2: Tool Integration and API Calling

The second milestone focused on extending the functionality of the conversational agent using external tools and APIs. LangChain tool abstraction mechanisms were studied and integrated into the workflow. Tools such as calculator and simulated weather APIs were implemented and connected with the agent system. Prompt engineering, tool invocation, API response handling, and error handling mechanisms were also implemented during this phase.

Milestone 3: Multi-Agent Orchestration and Memory Management

The third milestone focused on implementing multi-agent collaboration and memory-based reasoning. Specialized agents such as research and summarizer agents were designed for handling different operations within the workflow. Communication between agents was implemented using chained execution mechanisms and shared memory systems. Conversation memory and vector-based memory mechanisms were also integrated to improve contextual understanding and workflow continuity.

Milestone 4: Complex Workflow Automation and Evaluation

The final milestone focused on implementing complex workflow automation using multiple agents, tools, APIs, and frontend interaction mechanisms. Multi-step workflows such as research, summarization, and automated task execution were developed using orchestration logic. REST APIs were implemented using Flask-based backend integration, and frontend interfaces were developed for user interaction. Testing, optimization, code cleanup, documentation, and workflow evaluation were also completed during this phase.

Through these milestones, practical understanding was gained about:

- LangChain workflow development,
- intelligent query routing,
- multi-agent communication,
- tool integration,
- Retrieval-Augmented Generation (RAG),
- memory management systems,

- backend API development,
- frontend interaction,
- workflow automation techniques.

Milestone	Description	Duration
M1	Environment Setup & Basic Agent Creation	17-02-2026 to 27-02-2026
M2	Tool Integration & API Calling	27-02-2026 to 11-03-2026
M3	Multi-Agent Orchestration & Memory Management	11-03-2026 to 21-03-2026
M4	Complex Workflow Automation, Evaluation and Frontend using HTML, CSS, JS, Flask	21-03-2026 to 05-04-2026

Table 1.1: Internship Milestone Timeline

The internship also improved practical skills related to debugging, modular system design, workflow optimization, API integration, documentation, and real-world AI application development using modern frameworks and technologies.

1.9 Methodology Overview

The project follows a workflow-based orchestration methodology for processing user queries using intelligent routing, specialized execution pipelines, AI agents, memory systems, and external tool integration. The methodology is designed to improve response quality, workflow organization, and contextual understanding during query execution.

The workflow begins when the user submits a query through the frontend interface [1]. The input query is first analysed by the routing mechanism, where the system identifies the query type, complexity level, and required operations. Based on this analysis, the query is directed to the most suitable execution pipeline.

The framework consists of five execution pipelines named SIMPLE, TOOL, CONTENT, COMPLEX, and DOC. Each pipeline is designed for handling a specific category of tasks. Simple conversational queries are processed through the SIMPLE pipeline, while queries requiring external tool usage are processed through the TOOL pipeline. The CONTENT pipeline handles structured content generation tasks, and the COMPLEX pipeline manages advanced reasoning and multi-step workflow execution. The DOC pipeline processes uploaded documents and performs document-based querying using Retrieval-Augmented Generation (RAG) [5].

After pipeline selection, specialized AI agents are activated for performing operations such as routing, research, analysis, summarization, validation, and response generation. Different agents communicate with each other to execute tasks collaboratively within the orchestration workflow.

When required, the system invokes integrated external tools such as calculator utilities, weather APIs, web search modules, unit converters, and system time services. These tools help the platform perform real-time operations dynamically during workflow execution. The framework also uses memory management mechanisms for maintaining contextual continuity and improving response relevance. Conversation memory and FAISS vector-based retrieval systems are used to store and retrieve contextual information from previous interactions and uploaded documents [6].

Finally, the processed output is formatted and returned to the user through the frontend dashboard interface. The overall methodology enables the system to manage conversational processing, workflow automation, document retrieval, and intelligent task execution within a unified multi-agent architecture.

TECHNOLOGY STACK AND TRAINING

Modern AI applications require multiple technologies for implementing intelligent workflows, conversational systems, memory management, backend communication, document processing, and deployment. The development of the multi-agent orchestration framework involved the integration of various frameworks, libraries, APIs, and storage mechanisms for building a scalable and workflow-oriented AI platform.

The project was mainly developed using Python because of its simplicity, extensive AI ecosystem, and strong community support. Flask was used for backend API development and workflow management, while LangChain was implemented for building intelligent agent workflows, prompt pipelines, memory systems, and tool integration mechanisms. Large Language Models were integrated using the Groq API with the LLaMA 3.1 model for high-speed inference and conversational processing.

The framework also uses Retrieval-Augmented Generation (RAG) and FAISS vector databases for document retrieval and persistent memory management. Multiple frontend technologies such as HTML, CSS, and JavaScript were used for developing the dashboard and user interaction layer. SQLite was implemented for storing chat history, analytics, and workflow-related information. In addition, Docker containerization and Hugging Face Spaces deployment were used for creating a portable and web-accessible AI orchestration environment. The combination of these technologies helped in developing an intelligent, modular, and scalable multi-agent system capable of handling conversational workflows, external tools, document processing, and memory-based contextual reasoning.

2.1 Python

Python was selected as the primary programming language for the development of the project because of its simplicity, readability, and extensive support for Artificial Intelligence and backend application development. Python provides a large ecosystem of libraries and frameworks that simplify the implementation of machine learning models, conversational AI systems, API development, workflow automation, and data processing applications. One of the major reasons for selecting Python is its strong support for AI and Natural Language Processing frameworks such as LangChain,

Hugging Face Transformers, FAISS, sentence-transformers, and machine learning libraries. These libraries help developers implement intelligent workflows, agent orchestration systems, vector databases, and document retrieval mechanisms efficiently.



Fig 2.1: Python

Fig 2.1 shows the Python, Python also provides better development flexibility for integrating external APIs, tool execution modules, database operations, and backend services within a single environment. The language supports rapid prototyping and modular application development, which makes it suitable for experimental and workflow-based AI systems.

Another important advantage of Python is its large developer community and extensive documentation support. Community-driven libraries, tutorials, and open-source resources help simplify development, debugging, optimization, and framework integration processes. This makes Python one of the most widely used languages in modern Artificial Intelligence and automation projects.

The project was developed using Python 3.10+, which provides several modern programming features and performance improvements. Features such as improved type hinting, structural pattern matching, enhanced error handling, and optimized asynchronous operations helped improve workflow implementation and code organization within the orchestration framework.

Python was also used for: backend API development, multi-agent workflow execution, prompt handling, memory management, vector database integration, document processing and deployment automation. The flexibility and scalability provided by Python helped in building an intelligent orchestration framework capable of combining conversational AI, workflow routing, memory systems, and external tool integration within a unified platform [10].

2.2 Flask Web Framework

Flask was used as the backend web framework for developing the REST API and workflow management system of the project. Flask is a lightweight Python micro-framework that provides flexibility for building scalable web applications, API services, and backend workflow systems. Its modular architecture and simple design make it suitable for AI-based applications and orchestration platforms Fig 2.2 shows the Flask.



Fig 2.2: Flask

In the project, Flask was mainly used for handling backend communication between the frontend dashboard, AI agents, APIs, memory systems, and database modules. The framework manages incoming user requests, query processing, workflow execution, and response generation within the orchestration system.

Flask routing mechanisms were implemented for handling different API endpoints and frontend interactions. Routes were created for operations such as:

- user query processing,
- document uploads,
- analytics handling,
- workspace management,
- memory retrieval,
- chat history operations.

The framework was also used for session management and user interaction handling. Session-based mechanisms helped maintain workflow continuity and user-specific conversational data during interactions with the orchestration platform. File upload functionality was implemented using Flask for processing uploaded PDF, DOCX, CSV, XLSX, and PPTX documents. Uploaded files were transferred to the document processing pipeline where text extraction, chunking, embedding generation, and vector storage were performed for Retrieval-Augmented Generation (RAG).

Flask was integrated with the SQLite database for storing: user information, chat history, analytics data, workflow records, and memory-related information. The framework also supported API integration with external services and intelligent workflow execution modules. The lightweight architecture, flexibility, and scalability provided by Flask helped in building an efficient backend system [3].

2.3 Conversational AI and Large Language Models (LLMs)

Conversational Artificial Intelligence refers to systems capable of understanding user queries and generating human-like responses through Natural Language Processing techniques. Modern conversational AI systems use Large Language Models (LLMs) for performing tasks such as question answering, reasoning, content generation, summarization, workflow execution, and intelligent interaction [4].

Large Language Models are deep learning models trained on large-scale text datasets collected from books, websites, research documents, articles, and other textual sources. These models learn language patterns, sentence structures, contextual relationships, and semantic understanding during the training process. Because of large-scale training, LLMs are capable of generating context-aware and human-like responses for different categories of tasks.

Most modern LLMs are based on Transformer architecture, which uses attention mechanisms for understanding relationships between words and maintaining contextual understanding during response generation. Transformer-based models provide improved scalability, parallel processing, and contextual reasoning compared to traditional sequential neural network architectures.

The project integrates the Groq API for accessing Large Language Model services. Groq provides high-speed inference capabilities optimized for real-time conversational AI applications. The orchestration framework uses the LLaMA 3.1-8b-instant model for processing conversational workflows, routing operations, content generation, and reasoning tasks.

The LLaMA 3.1-8b-instant model is designed for fast inference and efficient response generation while maintaining contextual understanding for complex conversational workflows. The model was integrated into the orchestration framework using API-based communication mechanisms for handling user interactions and multi-agent execution pipelines. Groq was selected because of its ultra-fast inference performance powered by specialized LPU (Language Processing Unit) hardware

architecture. Faster inference improves workflow execution speed and reduces response latency during multi-agent communication and tool-based operations [2].

The project also uses temperature parameters during response generation. The temperature setting controls the randomness and creativity of generated responses. Lower temperature values produce more deterministic and consistent outputs, while higher temperature values generate more creative and diverse responses. Different temperature values were used depending on the workflow requirements and response generation tasks within the orchestration.

2.4 LangChain Framework

LangChain is an open-source framework designed for developing applications powered by Large Language Models (LLMs). The framework provides tools and components for building intelligent conversational systems, multi-agent workflows, memory-based applications, document retrieval systems, and tool-integrated AI platforms. LangChain simplifies the process of connecting language models with prompts, APIs, memory modules, external tools, and workflow execution pipelines.

In this project, LangChain was used as the core orchestration framework for implementing intelligent query routing, agent communication, prompt handling, memory management, tool execution, and Retrieval-Augmented Generation (RAG). The framework helped organize the system into modular workflows where different agents and pipelines perform specialized tasks during execution.

One of the major reasons for selecting LangChain is its support for agent-based architectures and workflow orchestration. The framework allows developers to create intelligent agents capable of reasoning, tool selection, memory retrieval, and task execution dynamically based on user queries. This capability was important for implementing the multi-agent orchestration architecture used in the project.

LangChain also provides built-in support for:

- prompt engineering,
- conversational memory,
- vector databases,
- document loaders,
- external tool integration,
- chain-based workflow execution.

These features simplified the development of the orchestration framework and reduced

the complexity of integrating multiple AI components within a single system.

The framework was also used for implementing:

- query routing pipelines,
- conversational workflows,
- document retrieval systems,
- semantic search operations,
- tool execution mechanisms,
- memory-aware response generation.

Several core LangChain components were used throughout the project for building the orchestration framework, agent communication layers, memory systems, and Retrieval-Augmented Generation workflows [1].

2.4.1 Prompt Templates

Prompt templates were used in the project for creating structured instructions for different AI agents within the orchestration framework. LangChain provides the `PromptTemplate` class, which helps generate dynamic prompts by combining predefined instructions, contextual information, and user queries before sending them to the Large Language Model. This approach improves workflow organization and helps the system generate more accurate and context-aware responses.

In the orchestration framework, each specialized agent was designed with its own unique prompt template according to the assigned functionality and execution pipeline. Instead of using a single generic prompt for every operation, separate prompt structures were created for different agents to improve specialization and workflow accuracy. The framework contains multiple agents responsible for operations such as routing, research, content generation, summarization, debugging, document analysis, response validation, and workflow coordination.

The Router Agent uses prompt templates focused on identifying query types and selecting the appropriate execution pipeline. Research-oriented agents use prompts designed for information retrieval, tool invocation, and reasoning-based operations. Summarization agents use prompt structures optimized for concise and context-aware response generation, while debugging agents use prompts focused on code analysis and error explanation.

Prompt templates also control the reasoning style, response formatting, contextual handling, and execution behaviour of different agents. Dynamic variables

such as user queries, memory content, retrieved document chunks, and workflow-specific instructions are inserted into the templates before processing. This helps the system maintain contextual continuity and generate workflow-aware responses across different pipelines. Another important advantage of using prompt templates is modularity within the orchestration framework. Since each workflow uses separate prompt structures, the system becomes easier to manage, optimize, and extend. Additional agents and workflows can be integrated without affecting the overall architecture of the system.

The use of structured prompt engineering techniques improved response consistency, workflow specialization, contextual understanding, and communication between different agents operating inside the orchestration framework [1].

2.4.2 Chains (LLMChain)

LangChain provides the concept of chains for implementing structured and sequential workflow execution between prompts, language models, and generated outputs. In the project, LLMChain was used as one of the core components for processing tasks that require controlled prompt execution and formatted response generation.

An LLMChain establishes a predictable processing pipeline where the user query is first combined with a structured prompt template, then passed to the Large Language Model for processing, and finally converted into the generated response. The execution workflow of the chain can be represented as:

User Input → PromptTemplate → Large Language Model → Structured Output

The use of LLMChain helped organize the orchestration framework into modular and task-specific processing units. Different chains were implemented for agents that required deterministic execution rather than autonomous tool selection. Each chain used its own prompt template and workflow logic according to the assigned responsibility of the agent.

In the project, LLMChain was implemented for several specialized agents such as the Content Agent, Medical Agent, Debug Agent, Summarizer Agent, and routing supervisors. The Content Agent used chain-based processing for generating structured content such as explanations, formatted documents, and contextual responses. The Medical Agent used controlled chains for generating organized medical-related responses with proper formatting and safety-oriented instructions. Similarly, the Debug Agent used LLMChain for processing code-related queries, analysing possible errors,

identifying root causes, and generating debugging explanations. Routing and classification supervisors also used chain-based execution for identifying query types and returning structured classification tags for downstream workflow routing.

The implementation of LLMChain improved workflow organization, response consistency, modularity, and execution control within the orchestration framework. Since each chain operates independently using dedicated prompts and structured processing logic, the system becomes easier to manage, optimize, and extend for additional workflows and specialized operations [1].

2.4.3 Agents (ReAct Framework)

Agents were one of the core components used in the orchestration framework for implementing intelligent workflow execution and dynamic decision-making. Unlike traditional conversational systems, agents are capable of analyzing user queries, selecting appropriate workflows, invoking external tools, processing contextual information, and generating task-specific responses dynamically.

The project uses the ReAct (Reasoning and Acting) framework provided by LangChain for implementing intelligent agent behavior. The ReAct approach combines reasoning and action-based execution, allowing the system to perform operations step-by-step instead of generating direct static responses.

The agents in the project were initialized using LangChain's `initialize_agent` function with the `ZERO_SHOT_REACT_DESCRIPTION` agent type. This configuration allows the agents to understand tool descriptions dynamically and decide which operation or tool should be used based on the user query.

One of the major implementations of the ReAct framework in the project is the Research Agent. This agent autonomously decides whether the query requires operations such as web searching, mathematical calculations, weather information retrieval, or other external tool execution. Based on the query type and workflow instructions, the agent selects the required tool and processes the operation dynamically. The orchestration framework follows a reasoning-based execution workflow in which the system first analyses the user query, identifies the required pipeline, activates the appropriate agents, retrieves contextual information from memory System when necessary, invokes external tools if required, and finally generates the response. The workflow can be represented as:

Analyze → Route → Execute → Retrieve → Generate Response

This execution mechanism helped improve workflow flexibility, intelligent routing, dynamic tool usage, and contextual reasoning within the orchestration framework. The use of agents also improved modularity and allowed different workflows to operate independently across multiple execution pipelines[1].

2.4.4 Memory Systems

To overcome the limitations of stateless Large Language Model APIs and maintain context-aware conversational workflows, the project implements a dual-layered memory architecture. The memory system is designed to balance immediate conversational continuity with long-term semantic context retrieval for document-based querying and personalized interactions.

The orchestration framework uses both short-term conversational memory and long-term vector-based semantic memory to improve contextual understanding during workflow execution. This architecture helps the system maintain conversation flow within active sessions while also retrieving semantically relevant information from previously processed documents and stored interactions.

The memory workflow in the system begins when the user submits a query to the Router Agent. Based on the query type, the system accesses either short-term conversational memory or long-term semantic memory. The retrieved contextual information is then passed to the language model before generating the final response.

Short-Term Memory: ConversationBufferMemory

The primary conversational workflow utilizes ConversationBufferMemory for maintaining short-term chat context during active user sessions. The memory system stores the chronological sequence of interactions between the user and the system during the current conversation session.

This mechanism helps the framework preserve conversational continuity and maintain contextual understanding during multi-turn interactions. The model can understand references from previous messages and maintain natural conversation flow without repeatedly asking for the same information.

The implementation of short-term memory improves:

- conversational continuity,
- session-level context retention,
- response relevance,
- interaction consistency during active sessions.

Long-Term Semantic Memory: VectorStoreRetrieverMemory

For document retrieval and long-term semantic recall, the project implements VectorStoreRetrieverMemory using the FAISS vector database. This mechanism enables the system to retrieve semantically relevant contextual information from uploaded documents and stored memory embeddings.

Uploaded files such as PDF, DOCX, TXT, CSV, XLSX, and PPTX documents are first processed through text extraction pipelines. The extracted content is divided into semantic chunks using text-splitting mechanisms with predefined chunk sizes and overlap values for improving retrieval accuracy.

After chunking, the text segments are converted into dense vector embeddings using the all-MiniLM-L6-v2 sentence-transformer model. These embeddings are stored inside FAISS vector databases for semantic retrieval operations. When the user submits a query, the system converts the query into an embedding vector and performs semantic similarity search within the vector database. Relevant contextual chunks are then retrieved and injected into the prompt before response generation.

The implementation of vector-based memory improved:

- document understanding,
- semantic retrieval,
- contextual response generation,
- personalized interactions,
- long-term conversational continuity within the orchestration framework [1].

2.5 RAG and Vector Databases

Retrieval-Augmented Generation (RAG) is an advanced AI architecture that combines information retrieval with Large Language Model response generation. Traditional Large Language Models generate responses only from the data used during model training and cannot naturally access uploaded files or dynamically changing information sources.

To overcome this limitation, the project implements a Retrieval-Augmented Generation architecture. Instead of relying only on the pre-trained knowledge of the language model, the system retrieves relevant contextual information from uploaded documents and injects it into the prompt before generating the response. This approach improves factual accuracy, contextual understanding, and document-based reasoning. long-term conversational continuity within the orchestration framework.

When users upload documents such as PDF files, Word documents, spreadsheets, presentations, or CSV files, the system processes the document content through extraction, chunking, embedding generation, and vector indexing mechanisms. During query execution, the framework retrieves only the most relevant contextual segments from the document instead of sending the entire file to the language model. This improves performance, reduces latency, and avoids exceeding model context limitations.

The project also implements semantic retrieval techniques instead of traditional keyword-based searching. Semantic retrieval converts text into high-dimensional vector representations and measures conceptual similarity between queries and stored document content. This allows the system to retrieve contextually relevant information even when the exact keywords are not present in the document.

Another major advantage of the RAG architecture is hallucination reduction. Large Language Models sometimes generate factually incorrect but confident responses when sufficient context is unavailable. The project reduces this problem by grounding responses using retrieved document excerpts. When a document-related query is detected, the orchestration framework routes the workflow to the document-processing pipeline and forces the response generation process to use retrieved contextual information instead of generating unsupported assumptions [5].

2.5.1 Document Embeddings

To perform semantic retrieval operations, the project converts raw document text into dense vector embeddings suitable for mathematical similarity comparison. Uploaded files are first processed through text extraction pipelines and divided into smaller overlapping segments using recursive text splitting mechanisms.

The framework uses recursive character-based text splitting with a chunk size of 1500 characters and a chunk overlap of 200 characters. Overlapping chunks help preserve semantic continuity. After chunking, the textual segments are converted into dense vector embeddings using the all-MiniLM-L6-v2 sentence-transformer model provided through Hugging Face. This model transforms semantic meaning into 384-dimensional numerical vector representations that can be compared mathematically during similarity search operations. The embeddings model was configured for efficient execution and optimized loading during application startup to improve performance and reduce runtime latency during document processing operations [5],[10].

2.5.2 FAISS Vector Search

After generating document embeddings, the vectors are stored using FAISS (Facebook AI Similarity Search), an open-source vector database library optimized for fast similarity search and nearest-neighbour retrieval operations. FAISS enables the orchestration framework to retrieve semantically relevant document chunks efficiently during user queries.

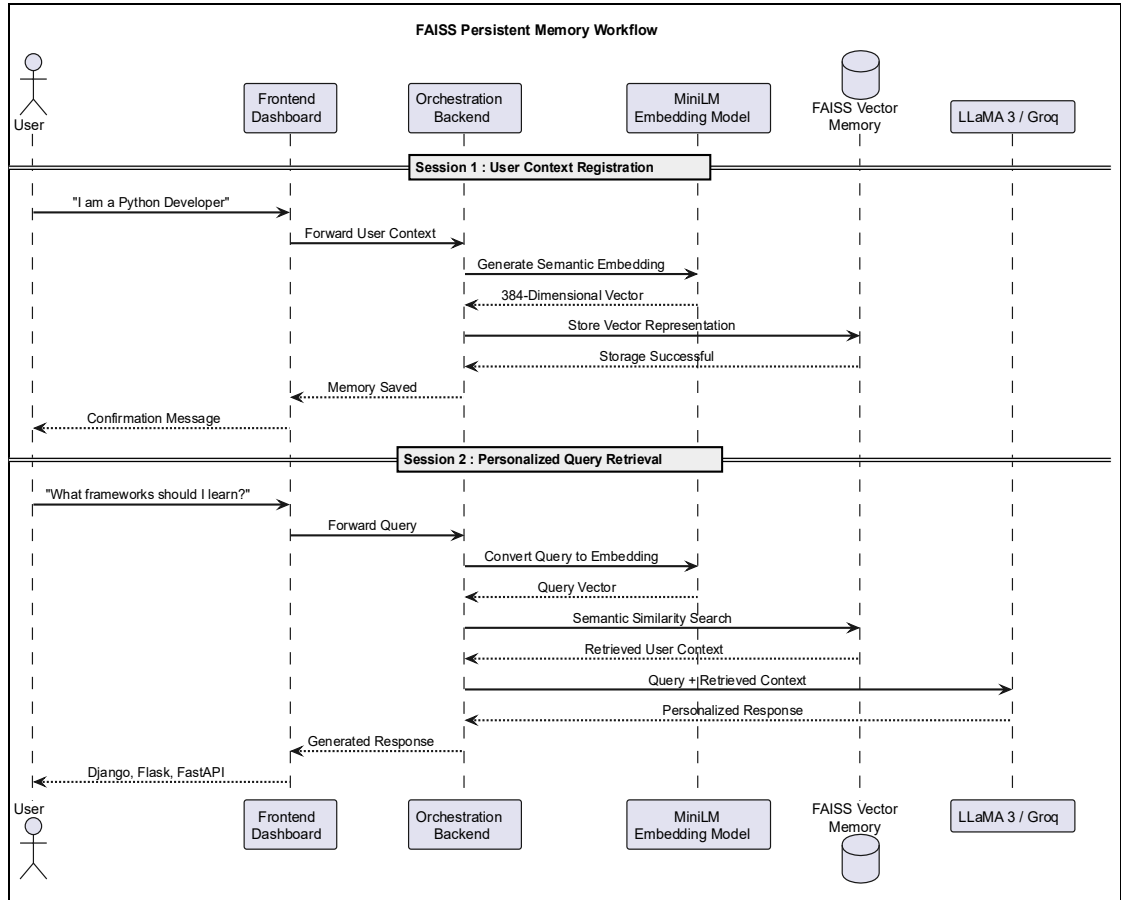


Fig 2.3: FAISS Persistent Memory Workflow

The vector database architecture was designed using isolated workspace-based storage mechanisms for maintaining independent document retrieval environments. Separate vector indexes are maintained for different workspaces and sessions to improve data organization and contextual separation.

When users upload new files, the system appends the new embeddings into the existing FAISS index instead of rebuilding the entire vector database. This improves performance and reduces computational overhead during incremental document ingestion operations. During query execution, the user query is converted into the same embedding vector space used during document ingestion. The FAISS database then performs similarity search operations to identify the most relevant contextual chunks

based on vector similarity scores. The top matching document segments are retrieved and injected into the prompt as contextual evidence before final response generation.

The implementation of FAISS vector retrieval significantly enhanced the ability of the system to perform semantic document analysis and context-aware information retrieval. It improved the accuracy of document-based querying by enabling the framework to retrieve semantically relevant content instead of relying on traditional keyword matching techniques. The vector retrieval mechanism also strengthened long-term contextual memory handling and helped generate more factually grounded responses within the orchestration framework [6].

2.6 Multi-Agent Systems

In Artificial Intelligence, a Multi-Agent System (MAS) refers to an architecture where multiple intelligent agents interact with each other to solve complex tasks collaboratively. Instead of relying on a single generalized model for every operation, the workload is distributed across multiple specialized agents, where each agent performs a dedicated task within the overall workflow. This approach improves modularity, reasoning capability, task specialization, and execution efficiency.

Traditional single-agent conversational systems often struggle when handling multiple responsibilities simultaneously, such as query routing, external tool execution, reasoning, content generation, document retrieval, and response validation. As the complexity of instructions increases, the system may experience instruction drift, reduced contextual consistency, and inefficient workflow execution. To overcome these limitations, the project implements a multi-agent orchestration architecture where the workload is distributed across multiple specialized AI agents operating under a centralized routing and supervision framework. Each agent was designed for a specific functionality and execution responsibility within the orchestration pipeline.

The architecture organizes the agents into multiple operational layers. The orchestration and routing layer is responsible for intercepting user queries and identifying the most suitable execution pipeline. Routing agents classify queries into categories such as document processing, tool execution, content generation, or complex reasoning workflows. Specialized domain agents then perform task-specific operations such as medical reasoning, debugging, research processing, content generation, and analytical workflows. The orchestration framework also includes a synthesis and quality-control layer responsible for response refinement, summarization, validation,

and optimization. Before generating the final response, the output passes through evaluation and refinement mechanisms to improve accuracy, formatting, and contextual consistency.

One of the major advantages of the multi-agent architecture is agent specialization. Instead of assigning all responsibilities to a single model, each agent operates within a restricted domain and follows dedicated prompt instructions. This improves task precision, reduces reasoning conflicts, and generates higher-quality responses for specialized workflows.

The orchestration framework also supports workflow orchestration and distributed execution. Complex user queries are divided into smaller analytical operations and processed through independent execution pipelines. Different agents and tools can operate simultaneously before the results are consolidated into the final response. This improves execution efficiency and reduces response latency during complex operations.

Another important advantage of the architecture is iterative quality control. Instead of directly displaying raw model outputs, the system performs response evaluation and refinement through multiple verification stages. If the generated output does not satisfy the validation requirements, refinement agents improve the response before delivery to the user. This mechanism improves output quality, consistency, and factual reliability within the orchestration framework. This improves task precision, reduces reasoning conflicts, and generates higher-quality responses for specialized workflows [4].

2.7 Frontend Technologies

The frontend layer of the project was developed using HTML, CSS, and JavaScript for creating an interactive and user-friendly dashboard interface. The frontend acts as the communication layer between the user and the orchestration backend, allowing users to interact with AI agents, upload documents, access analytics and view responses.

HTML was used for designing the structural layout of the application interface, including navigation sections, dashboard components, chat interfaces, forms, and workspace panels. The markup structure helped organize the frontend into modular and manageable UI sections.

CSS was used for improving the visual appearance and responsiveness of the application. Different styling techniques were implemented for layouts, navigation bars,

dashboard panels, buttons, chat windows, and analytics sections. The use of CSS improved interface consistency, readability, and user interaction across different pages of the application.

JavaScript was implemented for enabling dynamic frontend interactions and real-time communication with backend APIs. It allowed the interface to process user requests, display generated responses dynamically, update dashboard content without full-page reloads, manage document uploads, and retrieve stored conversation history efficiently. The use of asynchronous communication mechanisms improved frontend responsiveness and enhanced the overall user experience during workflow execution.

The project also includes a dashboard-based interface for managing conversations, uploaded files, analytics, search operations, pinned messages, and workspace activities. The dashboard UI was designed to simplify interaction with the orchestration framework and provide organized access to different system.

2.8 Database and Storage

The project uses SQLite as the primary database system for storing application data, workflow information, user interactions, and analytics records. SQLite was selected because of its lightweight architecture, easy integration with Flask, and suitability for AI-based applications that require efficient local storage management.

The database system maintains important information related to user sessions, workspace management, conversation records, analytics data, and orchestration workflows. It acts as the central storage layer for maintaining application consistency and preserving user interactions across different sessions. Chat history storage was implemented to maintain conversational continuity within the orchestration framework. The system stores previous interactions between the user and AI agents, allowing users to access past conversations and continue workflows without losing contextual information.

The project also maintains analytics-related storage for monitoring workflow execution and system activity. Different workflow routing, and user interaction are stored for dashboard visualization and performance tracking purposes. For contextual personalization and semantic memory retrieval, the framework integrates vector-based storage mechanisms using FAISS vector databases [6]. User interactions and processed document embeddings are stored as semantic vectors, enabling the system to retrieve contextually relevant information during future interactions. The storage architecture

was designed to support organized data management, memory retrieval, workflow synchronization, and efficient interaction handling within the orchestration framework.

2.9 Version Control and Deployment

Git and GitHub:

Git was used as the version control system for managing source code, tracking modifications, and maintaining project history throughout the development lifecycle. It helped organize the development workflow and simplified feature updates, debugging, and rollback operations during implementation.



Fig 2.4: Git and GitHub

GitHub was used as the centralized repository platform for storing the project source code and synchronizing development changes. The repository also helped maintain backup copies of the project and manage different stages of development efficiently.

Docker Containerization:

Docker was used for containerizing the application and creating a portable runtime environment for deployment. The orchestration framework, dependencies, libraries, APIs, and runtime configurations were packaged inside Docker containers to ensure consistent execution across different systems. Containerization also simplified environment setup and reduced dependency-related issues during deployment and testing operations [7].

Hugging Face Spaces Deployment:

The final application was deployed using Hugging Face Spaces, which provided a cloud-based hosting environment for the orchestration framework. The deployment platform enabled the application to be accessed through a web interface and allowed users to interact with conversational workflows, document-processing pipelines, and AI agents in real time.

The deployment process improved application accessibility, portability, and runtime management for the overall system [8].



Fig 2.5: Hugging Face

Deployment Workflow and Runtime Management:

The deployment workflow involved integrating the Flask backend, frontend dashboard, vector database storage, and AI orchestration modules into a unified runtime environment. Proper dependency management and deployment configuration ensured stable execution of conversational workflows, memory retrieval systems, external tools, and document-processing operations within the deployed application.

SYSTEM ANALYSIS AND DESIGN

System Analysis and Design is a crucial phase in the development of the Agent-Orchestration Framework with LangChain. It focuses on understanding system requirements and defining the overall architecture. The system is designed to process user queries through intelligent routing mechanisms. Multiple pipelines are used to handle different categories of requests efficiently. Specialized agents collaborate to perform reasoning, content generation, and document analysis. Memory management is implemented using FAISS and SQLite databases. Diagrams are used to represent workflows, data flow, and system interactions.

3.1 Requirement Specification

The requirement specification phase defines the operational capabilities and system constraints of the multi-agent orchestration framework. It ensures that the platform functions as an intelligent, scalable, and workflow-oriented AI system capable of handling conversational processing, document understanding, memory retrieval, and real-time tool execution. The requirement analysis also helps define how different modules interact within the orchestration architecture to provide efficient and context-aware response generation.

3.1.1 Functional Requirements

Functional requirements describe the major operations and feature that the system must perform during workflow execution and user interaction. The framework must be capable of analysing user queries and routing them dynamically to appropriate execution pipelines such as SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows. The routing mechanism should identify query types and activate suitable specialized agents for handling the requested operation.

The system must maintain persistent memory using vector databases for storing contextual information and semantic embeddings. It should retrieve relevant historical context during future interactions to improve personalization and contextual response generation. The orchestration framework must support real-time external tool integration for operations such as web searching, mathematical calculations, weather retrieval, unit conversion, and system time processing. The system must provide a dashboard-based user interface for managing conversations, uploaded files, chat

history, analytics, workspace operations, and pinned responses. It should also maintain conversational records and workflow data using database storage mechanisms.

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance expectations, and operational constraints of the orchestration framework. The system must provide fast response generation and efficient workflow execution during conversational processing. Query routing and lightweight preprocessing techniques should reduce unnecessary computational overhead and improve response latency.

The framework must be modular and scalable so that additional agents, workflows, tools, and memory components can be integrated without affecting the overall architecture of the system. The orchestration platform must maintain response accuracy and contextual consistency during multi-agent execution. Validation and refinement mechanisms should reduce hallucinations and improve the quality of generated responses before presenting them to the user.

The frontend interface must provide smooth user interaction and responsive dashboard behaviour during real-time conversational workflows. Dynamic communication between frontend and backend components should improve usability and reduce interface delays. The system must securely manage uploaded files, API configurations, and stored contextual data during workflow execution. Sensitive configuration details such as API credentials should be handled using secure environment configuration mechanisms.

The deployment environment must support stable runtime execution and portability across different systems. Containerized deployment mechanisms should simplify application hosting, dependency management, and runtime consistency within the orchestration framework.

3.2 Query Processing Techniques

The query processing layer is one of the most important components of the orchestration framework because it controls how user requests are analysed, classified, routed, and executed inside the system. The primary objective of this layer is to ensure that every user query is directed toward the most suitable execution pipeline and specialized agents based on the complexity and nature of the request. The framework follows a hybrid query-processing architecture that combines lightweight regex-based routing with semantic Large Language Model classification. This approach helps reduce

unnecessary computational overhead while maintaining intelligent workflow execution for complex queries. When a user submits a query through the frontend interface, the request first passes through the input analysis stage, where the system examines the user message, uploaded files, session information, and conversational context. After preprocessing, the framework performs regex-based pre-routing to identify common workflow categories quickly without invoking expensive LLM operations.

If the query cannot be confidently classified through regex matching, the request is forwarded to the LLM Router Agent for semantic understanding and advanced classification. The Router Agent then determines the most appropriate execution pipeline such as SIMPLE, TOOL, CONTENT, COMPLEX, or DOC workflows.

After pipeline selection, the corresponding agents, tools, memory systems, and retrieval mechanisms are activated for workflow execution. The generated outputs are finally processed through formatting and quality-control layers before being displayed to the user. This multi-stage query-processing architecture improves workflow efficiency, execution accuracy, contextual understanding, response quality, and overall orchestration performance within the system [1].

3.2.1 Input Analysis

The query processing workflow begins with the input analysis stage. In this stage, the orchestration framework receives the user query through the Flask backend using the POST /send_message API endpoint. The backend acts as the communication bridge between the frontend dashboard and the orchestration engine.

After receiving the request, the system analyses the query content, session information, workspace context, and uploaded file details before initiating workflow execution. This preprocessing stage helps the framework understand the nature of the request and prepare structured contextual input for further routing and execution. The framework also performs automatic file attachment detection during input analysis. If the user uploads documents along with the query, the system identifies the attached files and activates the document-processing workflow automatically.

Uploaded document files are passed to text extraction modules for content retrieval. The extracted textual information is then cleaned and prepared for semantic chunking, embedding generation, and retrieval-based processing. The input analysis layer ensures that both queries and uploaded document content are properly processed before entering the routing and orchestration stages of the framework.

3.2.2 Regex-Based Pre-Routing

To improve routing efficiency and reduce unnecessary Large Language Model computation, the framework implements a lightweight regex-based pre-routing mechanism before invoking semantic LLM routing. This preprocessing layer quickly identifies common query types and directly routes them to the appropriate execution pipelines without performing expensive language-model analysis.

The pre-routing system uses five predefined regex pattern categories for identifying workflow types:

- SIMPLE_PATTERNS
- TOOL_PATTERNS
- CONTENT_PATTERNS
- DOC_PATTERNS
- COMPLEX_PATTERNS

Each pattern category contains predefined expressions designed to recognize specific query behaviours and execution requirements. These regex groups help the framework classify user requests efficiently during the early stages of workflow processing.

The orchestration framework follows a priority-based routing order during regex evaluation. The routing sequence is defined as:

TOOL → DOC → SIMPLE → CONTENT → COMPLEX

This priority system ensures that tool-related operations and document-based queries are detected before lightweight conversational workflows and advanced reasoning operations. Queries requiring external tools such as weather retrieval, calculations, or web searching are immediately routed to the TOOL pipeline. Similarly, document-oriented requests are prioritized for retrieval-based processing through the DOC pipeline.

If the incoming query does not match any regex pattern category, the request automatically falls through to the LLM Router Agent for semantic classification and advanced workflow analysis. This hybrid routing mechanism balances execution speed with intelligent contextual understanding. The framework also includes a document-priority override mechanism. When relevant contextual information is retrieved from the vector retrieval system, the orchestration framework automatically forces the query into the DOC pipeline regardless of its original routing classification. This override remains active except when TOOL or CONTENT workflows are explicitly required.

The regex-based pre-routing layer significantly improves routing speed, workflow optimization, response latency, and computational efficiency within the orchestration framework.

3.2.3 LLM Router Classification

The orchestration framework uses an intelligent LLM-based routing mechanism for handling queries that cannot be accurately classified through regex-based pre-routing. This routing layer acts as the central decision-making component of the system and performs semantic analysis on the user query before selecting the appropriate execution workflow.

The Router Agent analyses the meaning, intent, and complexity of the incoming request using structured prompt engineering techniques. Based on semantic understanding, the system dynamically assigns the query to one of the major execution pipelines responsible for handling different categories of operations within the framework.

Queries related to general conversations, basic factual information, and lightweight interactions are directed toward the SIMPLE pipeline for fast response generation. Requests involving calculations, weather retrieval, web searching, unit conversion, and real-time operations are routed to the TOOL pipeline. Content-generation tasks such as email drafting, report writing, essay creation, storytelling, and structured text generation are assigned to the CONTENT pipeline. For advanced analytical and reasoning-oriented requests, the framework activates the COMPLEX pipeline. Inside this workflow, a secondary routing mechanism called the Use-Case Router further categorizes the query into specialized execution domains. This additional classification layer helps identify whether the request belongs to medical reasoning, decision analysis, debugging, shopping assistance, or general research operations. Based on this classification, the framework activates the corresponding specialized agents for task execution.

The orchestration framework also implements a document-priority routing mechanism integrated with the FAISS vector retrieval system. When semantically relevant document context is retrieved from uploaded files or stored vector memory, the routing process automatically prioritizes the DOC execution pipeline to ensure that contextual document information is incorporated during response generation. The query explicitly requires tool execution or content-generation workflows. The LLM Router

classification mechanism improves contextual understanding, workflow specialization, semantic routing accuracy, and intelligent orchestration across the multi-agent execution framework.

3.2.4 Output Formatting

After the execution pipeline completes the processing of the user query, the generated output passes through a dedicated formatting and quality-control stage before being delivered to the frontend interface. This layer is responsible for improving readability, maintaining response consistency, and ensuring professional presentation of the generated content.

The orchestration framework uses the Summarizer Agent to convert raw outputs, analytical data, research information, tool responses, and multi-agent results into structured Markdown-formatted responses. The formatting process organizes the generated content into clear sections, headings, bullet points, tables, and readable response structures to improve user interaction and presentation quality.

Following the formatting stage, the generated response is evaluated by the Confidence Agent. This agent performs quality verification by analysing the completeness, contextual accuracy, readability, and consistency of the generated output. The validation process helps identify incomplete explanations, formatting issues, ambiguous information, and low-confidence responses. If the generated response does not satisfy the required quality conditions, the framework automatically activates the Refinement Agent. This agent improves the response by restructuring explanations, enhancing clarity, correcting formatting inconsistencies, and refining the overall presentation before final delivery to the user.

The multi-stage output formatting architecture improves response quality, readability, contextual consistency, and user experience within the orchestration framework.

3.3 Proposed System Architecture

The proposed system architecture follows a layered multi-agent orchestration design that combines conversational AI, workflow routing, document processing, memory retrieval, external tool execution, and database management within a unified framework. The architecture was designed to support scalable query processing, intelligent workflow execution, contextual memory handling, and real-time interaction through specialized AI agents and orchestration pipelines. The framework is divided

into multiple operational layers, where each layer performs a dedicated responsibility within the overall execution workflow. The layered architecture improves modularity, maintainability, workflow isolation, and execution efficiency across different components of the system.

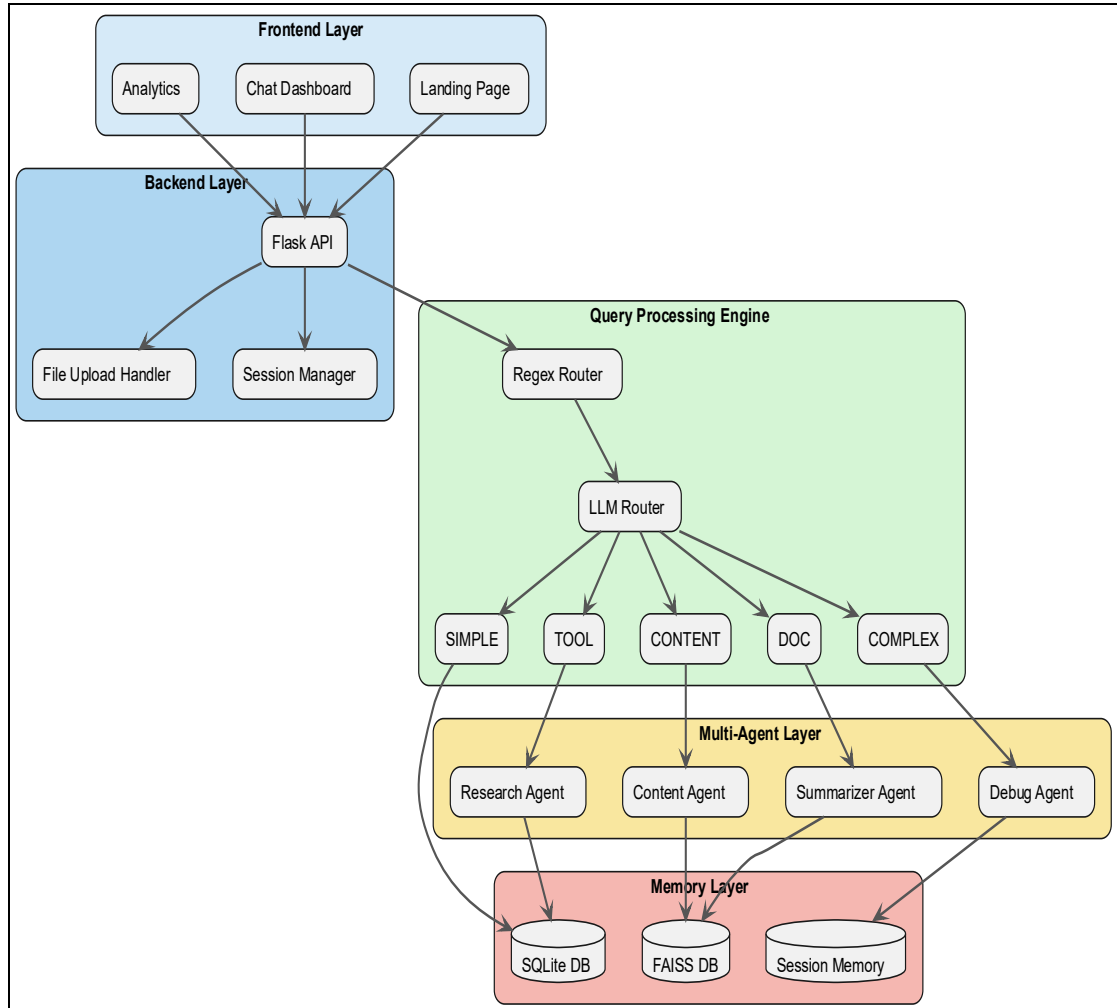


Fig 3.1: Overall System Architecture

The framework is divided into multiple operational layers, where each layer performs a dedicated responsibility within the overall execution workflow. The layered architecture improves modularity, maintainability, workflow isolation, and execution efficiency across different components of the system.

The User Interface Layer acts as the interaction point between the user and the orchestration framework. This layer includes modules such as the landing page, authentication pages, chat dashboard, analytics section, search functionality, history management, and workspace interfaces. Users interact with the system through these frontend components for conversational workflows, document uploads, and dashboard operations. The Flask Backend Layer acts as the central communication and API

management layer of the framework. This layer handles authentication, session management, API routing, file uploads, database interaction, and communication between frontend and orchestration modules. The Flask backend also manages request handling and transfers processed queries to the orchestration engine for workflow execution.

The Query Processing Engine forms the core execution controller of the system. This layer performs query preprocessing, regex-based routing, semantic LLM classification, pipeline selection, workflow orchestration, and execution management. The orchestration engine dynamically routes user queries into execution pipelines such as SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows depending on query requirements and contextual analysis.

The Multi-Agent Layer contains specialized AI agents responsible for handling domain-specific tasks within the orchestration framework. Different agents collaborate for routing, research, debugging, content generation, medical reasoning, summarization, quality evaluation, optimization, and response validation. This multi-agent execution model improves specialization, reasoning quality, and workflow coordination.

The Memory and Data Layer is responsible for persistent storage, conversational memory handling, vector retrieval, and database management. SQLite databases are used for storing chat history, user data, uploaded files, analytics information, and workspace records, while FAISS vector databases manage semantic memory retrieval and document-based contextual search. Session-based memory mechanisms also maintain short-term conversational context during workflow execution.

The layered system architecture improves scalability, modular workflow management, contextual intelligence, response quality, and efficient orchestration across the multi-agent AI framework.

3.4 Workflow Orchestration Pipelines

The orchestration framework uses a multi-pipeline execution architecture for handling different categories of user queries efficiently. Instead of processing every request through a single workflow, the system dynamically selects specialized execution pipelines based on query complexity, contextual information, routing analysis, and operational requirements. This architecture improves workflow specialization, response

quality, execution speed, and computational efficiency within the framework. The workflow execution process begins after query routing and classification are completed by the pre-routing layer and LLM Router Agent. Based on the identified query type, the orchestration framework activates one of the five major execution pipelines:

- SIMPLE Pipeline
- TOOL Pipeline
- CONTENT Pipeline
- COMPLEX Pipeline
- DOC Pipeline

Each pipeline was designed for handling a specific category of tasks within the orchestration framework.

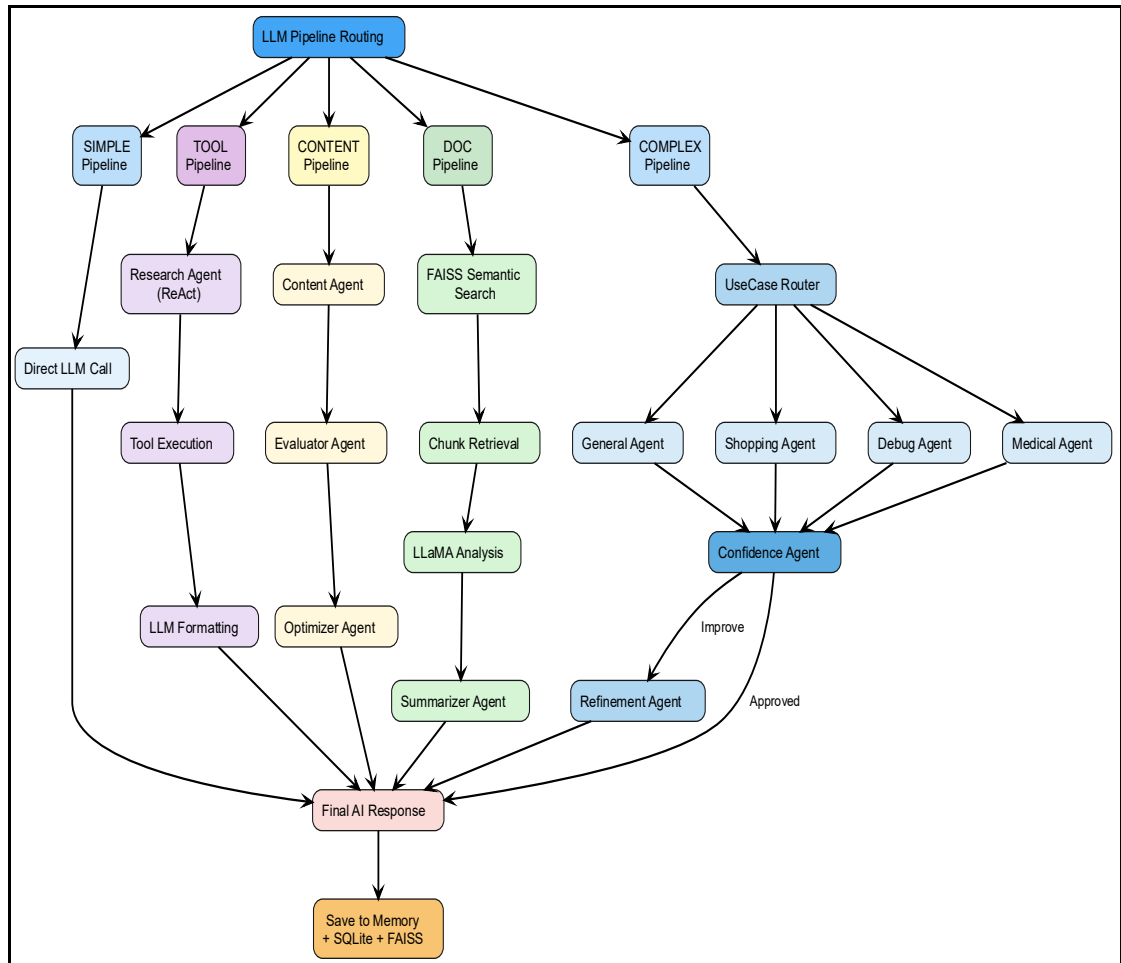


Fig 3.2: Pipeline Execution Workflow

Lightweight conversational queries are processed through fast-response workflows, while advanced reasoning operations activate collaborative multi-agent execution mechanisms. Similarly, document-related queries use semantic retrieval pipelines integrated with vector databases for contextual analysis.

The orchestration engine also supports dynamic pipeline switching, fallback execution handling, quality-control workflows, and contextual routing overrides during runtime processing. This allows the framework to manage different operational scenarios while maintaining workflow stability and response consistency.

The pipeline-based architecture improves modular execution, intelligent task distribution, agent specialization, and scalable orchestration across the AI framework.

3.4.1 SIMPLE Pipeline

The SIMPLE pipeline is the fastest execution workflow within the orchestration framework. This pipeline is designed for handling lightweight conversational queries that do not require external tools, document retrieval, advanced reasoning, or multi-agent collaboration. The primary objective of this workflow is to provide low-latency response generation while reducing unnecessary orchestration overhead.

When a user query is classified as a simple request through regex-based routing or semantic LLM classification, the orchestration framework directly invokes the Large Language Model without activating additional specialized agents or workflow pipelines. This direct execution mechanism bypasses complex processing stages and generates responses immediately using standard conversational inference.

The SIMPLE pipeline is primarily used for operations such as greetings, basic factual questions, definitions, general informational queries, and lightweight conversational interactions. Since these requests do not require tool execution or contextual retrieval, the framework avoids unnecessary routing complexity and prioritizes execution speed.

The direct LLM invocation architecture improves response latency, reduces computational cost, and increases system efficiency for high-frequency conversational interactions. By isolating lightweight workflows from advanced orchestration pipelines, the framework maintains balanced resource utilization and optimized query processing performance within the overall system.

3.4.2 TOOL Pipeline

The TOOL pipeline is responsible for handling queries that require external tool execution, real-time information retrieval, mathematical computation, and utility-based operations. This workflow enables the orchestration framework to interact with live external services and specialized processing utilities instead of relying only on static LLM-generated responses. When a query enters the TOOL pipeline, the framework

initially performs fast-path validation for common utility operations such as weather retrieval, current time tracking, mathematical calculations, unit conversion, and web searching. If the request directly matches predefined tool conditions, the tool is executed immediately for improving response speed and reducing unnecessary reasoning overhead [1].

The orchestration framework integrates multiple real-time tools including:

- weather retrieval services,
- advanced mathematical calculators,
- web search mechanisms,
- unit conversion utilities,
- and global time retrieval systems.

For more complex tool-based workflows, the system activates the Research Agent operating under the ReAct reasoning framework. In this execution model, the agent performs reasoning, selects the required tool autonomously, executes the operation, observes the result, and generates the final response based on the retrieved output. This dynamic execution strategy allows the framework to handle multi-step tool interactions intelligently.

The TOOL pipeline also implements fallback handling mechanisms for maintaining workflow stability. If the Research Agent fails to produce a valid response or encounters execution limitations during tool usage, the framework automatically switches to direct LLM-based response generation to ensure uninterrupted workflow execution. The TOOL pipeline improves real-time processing capability, execution flexibility, intelligent tool orchestration, and response accuracy within the multi-agent orchestration framework.

3.4.3 CONTENT Pipeline

The CONTENT pipeline is responsible for handling content-generation tasks within the orchestration framework. This workflow is used for generating structured textual outputs such as emails, essays, stories, reports, explanations, and source code generation requests. Unlike simple conversational workflows, the CONTENT pipeline follows a multi-stage quality-assurance architecture to improve response quality and formatting consistency. When a query is routed to the CONTENT pipeline, the Content Agent first generates the initial draft based on the user request and contextual information. The generated draft is then forwarded to the Evaluator Agent, which

performs quality assessment on the generated content. The evaluation process checks multiple factors such as formatting quality, grammatical consistency, structural organization, readability, and contextual relevance.

If the generated output does not satisfy the required quality conditions, the orchestration framework activates the Optimizer Agent. The Optimizer Agent rewrites and improves the generated content by refining sentence structure, improving clarity, correcting formatting issues, and enhancing overall presentation quality.

The CONTENT pipeline implements a cyclical quality-assurance loop in which the rewritten output can be re-evaluated for additional improvements. To maintain execution efficiency and avoid excessive processing overhead, the optimization workflow is limited to a maximum of two refinement iterations.

This multi-stage content-generation architecture improves professionalism, readability, formatting consistency, and response quality for generated textual outputs within the orchestration framework [1].

3.4.4 COMPLEX Pipeline

The COMPLEX pipeline is designed for handling advanced reasoning workflows that require multi-step analysis, contextual understanding, domain specialization, and collaborative agent execution. When a query enters this pipeline, the orchestration framework activates the Use-Case Router for secondary classification. The router analyses the semantic intent of the query and categorizes it into one of five specialized execution domains [4].

Medical Workflow

Medical-related queries are processed using the Medical Agent for performing health-related reasoning and informational analysis. The workflow provides structured responses related to symptoms, conditions, medications, and general medical guidance. In situations requiring deeper contextual verification, the Research Agent may perform additional specialist-level review before generating the final response. This workflow enables the framework to process medical queries in structured and controlled manner.

Decision Analysis Workflow

Decision-based queries activate a collaborative workflow involving the Research Agent, Decision Agent, and Summarizer Agent. The framework performs analytical comparison, evaluates alternatives, identifies advantages and disadvantages, and generates recommendation-oriented outputs for the user.

Debugging Workflow

Technical and programming-related queries are processed through the Debug Agent. This workflow analyses source code issues, identifies possible errors, detects root causes, and generates corrective solutions. If required, the framework may also perform optional web-search operations for retrieving additional debugging references and technical solutions.

Shopping Workflow

Shopping-related workflows use the Research Agent for collecting product information, online pricing data, reviews, and comparison details from external sources. The retrieved information is then analysed by the Shopping Agent before the Summarizer Agent generates a structured recommendation response.

General Analytical Workflow

The GENERAL workflow is responsible for handling broad analytical and multi-part reasoning tasks. The framework first performs multi-part query detection to identify complex reasoning objectives. It may then execute parallel research operations for gathering contextual information from multiple sources simultaneously. The retrieved data is processed through the General Agent for analytical reasoning before passing through the Summarizer Agent and Confidence Agent for quality validation. If additional refinement is required, the Refinement Agent improves the response before final delivery.

The COMPLEX pipeline improves reasoning capability, workflow specialization, collaborative execution, and intelligent orchestration across advanced analytical scenarios within the framework.

3.4.5 DOCUMENT Pipeline

The DOC pipeline is responsible for handling document-based conversational workflows within the orchestration framework. This pipeline is automatically activated whenever document context is available through uploaded files or semantic memory retrieval systems. The primary objective of this workflow is to enable Retrieval-Augmented Generation (RAG) by combining semantic document retrieval with Large Language Model reasoning for contextual response generation [5].

When a user uploads a document such as a PDF, DOCX, CSV, XLSX, PPTX, or TXT file, the framework first performs document loading and text extraction. The extracted textual content is then passed to the text-splitting module, where the document

is divided into smaller semantic chunks for efficient vector processing and retrieval operations. After chunk generation, the orchestration framework converts each document chunk into dense vector embeddings using transformer-based embedding models.

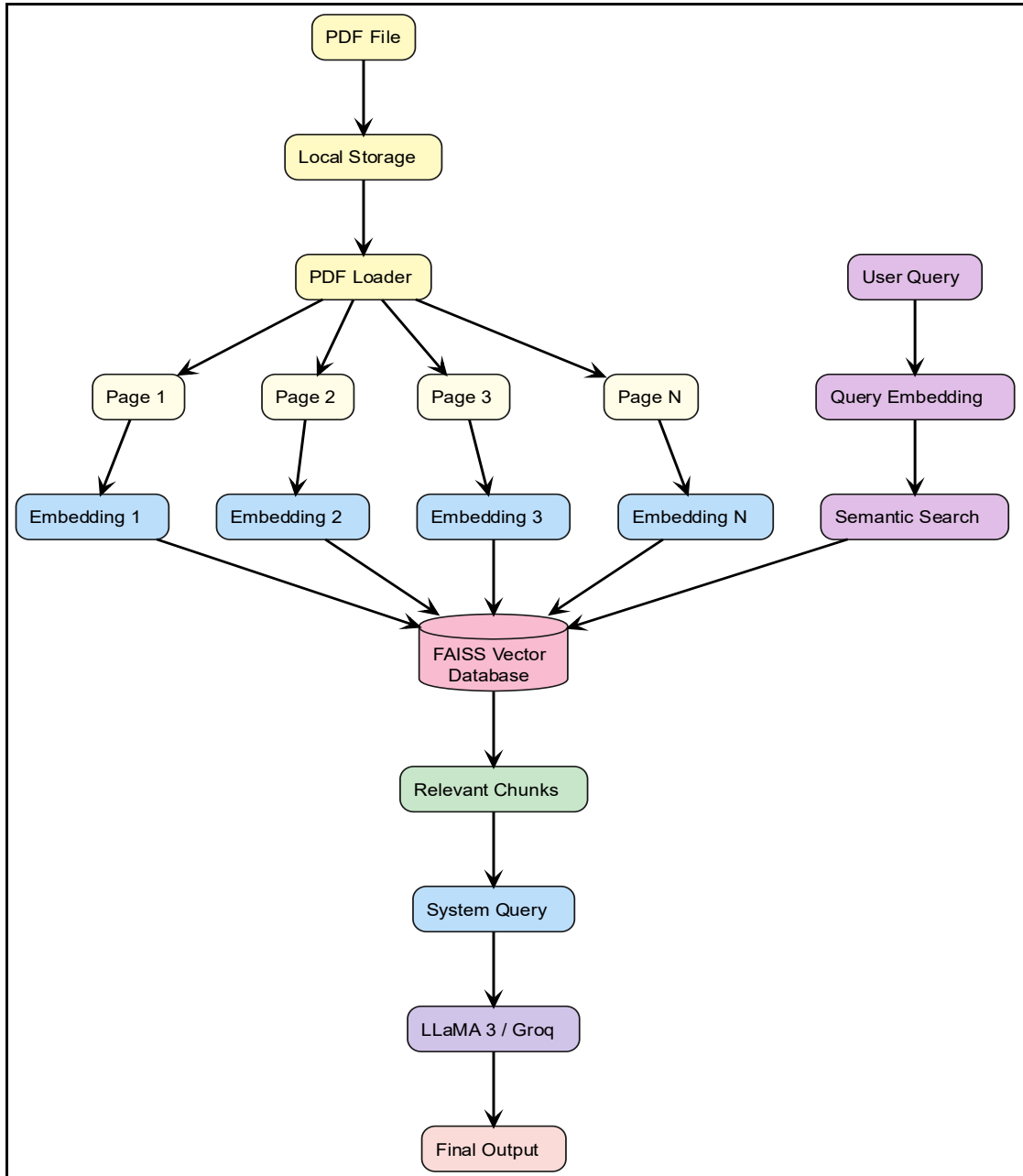


Fig 3.3: RAG-Based Document Processing Workflow

These embeddings are stored inside the FAISS vector database, which acts as the semantic retrieval engine for the DOC pipeline. During query execution, the user query is also converted into an embedding vector. The FAISS retriever then performs semantic similarity search between the query embedding and stored document embeddings to identify contextually relevant information. The system retrieves the top

matching document chunks, typically selecting the top-8 most relevant semantic results for contextual analysis. The retrieved document context is then injected into the orchestration workflow along with the user query before being forwarded to the Large Language Model for response generation. This Retrieval-Augmented Generation mechanism improves contextual understanding, factual grounding, and document-aware reasoning within the framework [6].

The DOC pipeline also supports contextual follow-up handling for vague conversational queries such as “tell me more,” “explain further,” or “continue.” In such cases, the framework uses previous conversational context and semantic retrieval mechanisms to identify related document sections automatically and continue contextual response generation without requiring the user to repeat the original request.

The orchestration framework additionally supports both broad and focused response-generation strategies within the DOC pipeline. For broad analytical queries, the system retrieves information from multiple document sections to generate comprehensive summaries and high-level explanations. For focused queries, the framework performs targeted semantic retrieval to provide precise responses related to specific sections, concepts, or contextual information within the uploaded document.

The DOC pipeline significantly improves document understanding, semantic retrieval accuracy, contextual memory handling, and conversational interaction with uploaded files within the orchestration framework.

3.5 Multi-Agent Architecture

The orchestration framework is based on a multi-agent architecture in which multiple specialized AI agents collaboratively process user requests instead of relying on a single conversational model. Each agent within the framework was designed for handling a specific category of operations such as routing, reasoning, debugging, content generation, summarization, validation, and quality assurance [1],[2].

The multi-agent design improves workflow specialization, modular execution, contextual reasoning, scalability, and response quality within the orchestration system. Instead of assigning all operations to one generic model, the framework distributes responsibilities across multiple intelligent agents based on query type and execution requirements. The orchestration engine dynamically activates different agents depending on the selected workflow pipeline. Lightweight queries may require only one agent, while advanced analytical workflows involve collaborative execution between

multiple agents operating sequentially or in parallel [4]. The framework internally categorizes the agents into five major architectural groups:

- Routing Agents
- Research and Analysis Agents
- Domain Expert Agents
- Content Creation Agents
- Quality Assurance Agents

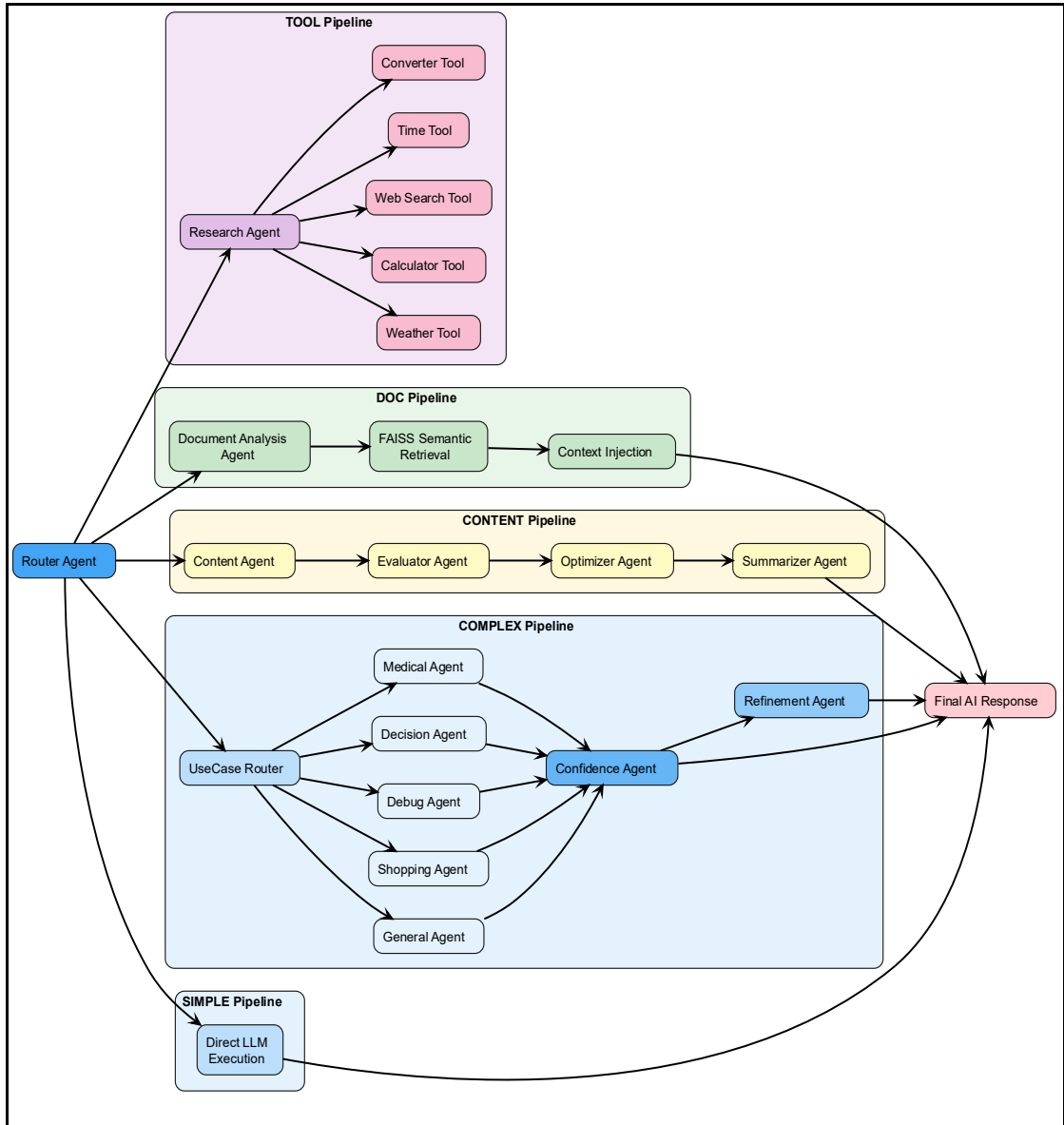


Fig 3.4: Multi-Agent Architecture Diagram

This modular architecture allows new agents, workflows, and reasoning mechanisms to be integrated into the system without affecting existing orchestration logic. The agent-based design also improves maintainability, extensibility, execution flexibility, and workflow optimization within the orchestration framework.

Sr. No.	Agent Name	Agent Category	Primary Responsibility	Pipeline Usage
1	Router Agent	Routing Agent	Classifies user queries into execution pipelines	All Pipelines
2	UseCase Router	Routing Agent	Sub-classifies COMPLEX queries into specialized domains	COMPLEX
3	Research Agent	Research & Analysis Agent	Performs research, tool selection, and contextual reasoning	TOOL, COMPLEX
4	Analysis Agent	Research & Analysis Agent	Performs analytical reasoning and contextual interpretation	COMPLEX
5	Medical Agent	Domain Expert Agent	Handles medical and healthcare-related reasoning	COMPLEX
6	Decision Agent	Domain Expert Agent	Performs comparison and recommendation analysis	COMPLEX
7	Debug Agent	Domain Expert Agent	Handles code debugging and technical troubleshooting	COMPLEX
8	Shopping Agent	Domain Expert Agent	Performs product analysis and recommendation workflows	COMPLEX
9	General Agent	Domain Expert Agent	Handles general analytical and reasoning workflows	COMPLEX
10	Content Agent	Content Creation Agent	Generates structured textual content and responses	CONTENT
11	Summarizer Agent	Content Creation Agent	Summarizes and formats large responses	CONTENT, COMPLEX, DOC
12	Evaluator Agent	Quality Assurance Agent	Evaluates response quality and contextual consistency	CONTENT
13	Optimizer Agent	Quality Assurance Agent	Refines and improves generated responses	CONTENT
14	Confidence Agent	Quality Assurance Agent	Performs final response validation and confidence checking	COMPLEX, CONTENT
15	Refinement Agent	Quality Assurance Agent	Performs additional response enhancement and correction	COMPLEX
16	Document Analysis Agent	Domain Expert Agent	Handles semantic document understanding and contextual analysis	DOC

Table 3.1: Complete Agent Registry

3.5.1 Routing Agents

Routing Agents are responsible for analysing user queries and selecting the appropriate execution workflow within the orchestration framework. These agents act as the central decision-making layer of the system and control how requests move across different pipelines, tools, and specialized agents.

The routing process begins immediately after input analysis and regex-based pre-routing operations. Initially, lightweight classification is performed using predefined routing patterns for identifying common query categories such as SIMPLE, TOOL, CONTENT, DOC, and COMPLEX requests. This fast-path routing mechanism reduces unnecessary computational overhead and improves response latency for straightforward queries. If the query cannot be confidently classified through regex-based matching, the orchestration framework activates the Router Agent. The Router

analytical workflows, the framework additionally uses a secondary routing component called the Use-Case Router. This routing layer operates within the COMPLEX pipeline and further categorizes requests into specialized execution domains such as:

- medical analysis,
- decision-making,
- debugging,
- shopping assistance,
- general analytical workflows.

The routing architecture also supports contextual override mechanisms. For example, when document context is available through semantic retrieval systems, the framework may automatically redirect the query to the DOC pipeline for Retrieval-Augmented Generation processing, even if the original classification belongs to another reasoning category. The Routing Agents improve execution efficiency, workflow specialization, intelligent task distribution, and dynamic orchestration control within the multi-agent framework.

3.5.2 Research and Analysis Agents

Research and Analysis Agents are responsible for collecting information, performing reasoning operations, analysing contextual data, and supporting decision-oriented workflows within the orchestration framework. These agents play a major role in handling dynamic queries that require multi-step reasoning, external information retrieval, and contextual interpretation.

The primary component within this category is the Research Agent, which operates using the ReAct reasoning methodology. In this execution model, the agent first analyses the user query, determines whether external tools are required, selects the appropriate tool dynamically, performs the required operation, observes the retrieved result, and then generates a context-aware response.

The Research Agent is heavily used in TOOL workflows, decision-analysis pipelines, shopping workflows, and advanced reasoning operations. It interacts with multiple external utilities such as web-search systems, weather services, mathematical calculators, time utilities, and conversion tools for retrieving real-time information during execution. The orchestration framework also supports analytical processing for complex multi-part queries. In such workflows, the system performs contextual information gathering, reasoning aggregation, and response synthesis before generating

the final output. Multiple research operations may execute sequentially or in parallel depending on the complexity of the query. Research and Analysis Agents improve real-time information processing, reasoning capability, contextual understanding, and intelligent decision support within the orchestration framework [1].

3.5.3 Domain Expert Agents

Domain Expert Agents are specialized AI agents responsible for handling queries that require domain-specific reasoning, contextual understanding, and task-oriented problem solving within the orchestration framework. Instead of relying on a single generalized conversational model, the framework distributes specialized operations across dedicated agents trained through prompt-engineering methodologies for handling particular execution domains.

The Medical Agent is responsible for processing health-related informational queries and medical reasoning workflows. This agent analyses symptoms, conditions, medications, treatment-related discussions, and healthcare-oriented user requests. The orchestration framework may additionally perform supplementary contextual verification through research-based workflows for improving response reliability in complex medical scenarios.

The Debug Agent handles programming-related operations such as code analysis, error detection, debugging workflows, exception analysis, and technical troubleshooting. The agent identifies possible root causes of software issues and generates corrective recommendations or optimized code solutions. In advanced debugging situations, the framework may also integrate external search mechanisms for retrieving additional technical references and implementation guidance.

The Shopping Agent processes product-oriented workflows involving price comparison, recommendation generation, feature evaluation, and product analysis. The external research operations for collecting real-time pricing information, reviews, and product details before generating structured recommendation responses.

The General Agent is responsible for handling broad analytical workflows that do not belong to a specific specialized domain. This agent processes contextual reasoning tasks, multi-part analytical questions, explanation-based requests, and generalized reasoning operations requiring dynamic contextual interpretation. The use of Domain Expert Agents improves workflow specialization, contextual intelligence, reasoning accuracy, and task-specific response quality within the framework[1].

3.5.4 Content Creation Agents

Content Creation Agents are responsible for generating structured textual outputs and formatted responses for different content-generation workflows within the orchestration framework. These agents support operations such as email drafting, essay generation, storytelling, report writing, summarization, explanation generation, and source-code creation.

The Content Agent generates the initial draft based on the user request, contextual information, execution pipeline, and workflow requirements. The generated content may include formal documents, descriptive explanations, structured reports, creative content, or technical outputs depending on the operational context.

The Summarizer Agent plays an important role in improving response readability and presentation quality. It converts large or complex outputs into concise and organized responses while preserving essential contextual information. The summarization workflow is heavily used in analytical pipelines, document-processing workflows, and research-oriented operations.

The content-generation architecture also supports formatting standardization for producing cleaner Markdown-based outputs and professionally structured responses. This improves consistency across different execution pipelines and enhances user readability within the conversational interface.

Content Creation Agents improve formatting quality, readability, content organization, and professional response generation within the orchestration framework.

3.5.5 Quality Assurance Agents

Quality Assurance Agents are responsible for validating generated responses and improving output quality before final response delivery. These agents act as the verification and refinement layer of the orchestration framework and help maintain response consistency, formatting quality, contextual accuracy, and execution reliability. The Evaluator Agent performs quality assessment on generated outputs by analysing grammatical structure, readability, contextual consistency, formatting quality, and logical coherence. This evaluation process is particularly important within content-generation workflows where professionally formatted outputs are required.

If the generated response does not satisfy quality conditions, the framework activates the Optimizer Agent. The Optimizer Agent rewrites and improves the generated output by refining sentence structure, improving clarity, correcting

formatting issues, and enhancing overall response presentation. The Confidence Agent performs final verification before the response is delivered to the user. This agent evaluates the reliability and confidence level of the generated response to determine whether additional refinement operations are required.

If the confidence evaluation identifies inconsistencies or quality limitations, the orchestration framework activates the Refinement Agent for performing additional enhancement and correction operations before generating the final response. The Quality Assurance architecture improves response reliability, contextual consistency, formatting quality, and overall execution stability within the orchestration framework.

3.6 Memory Management System

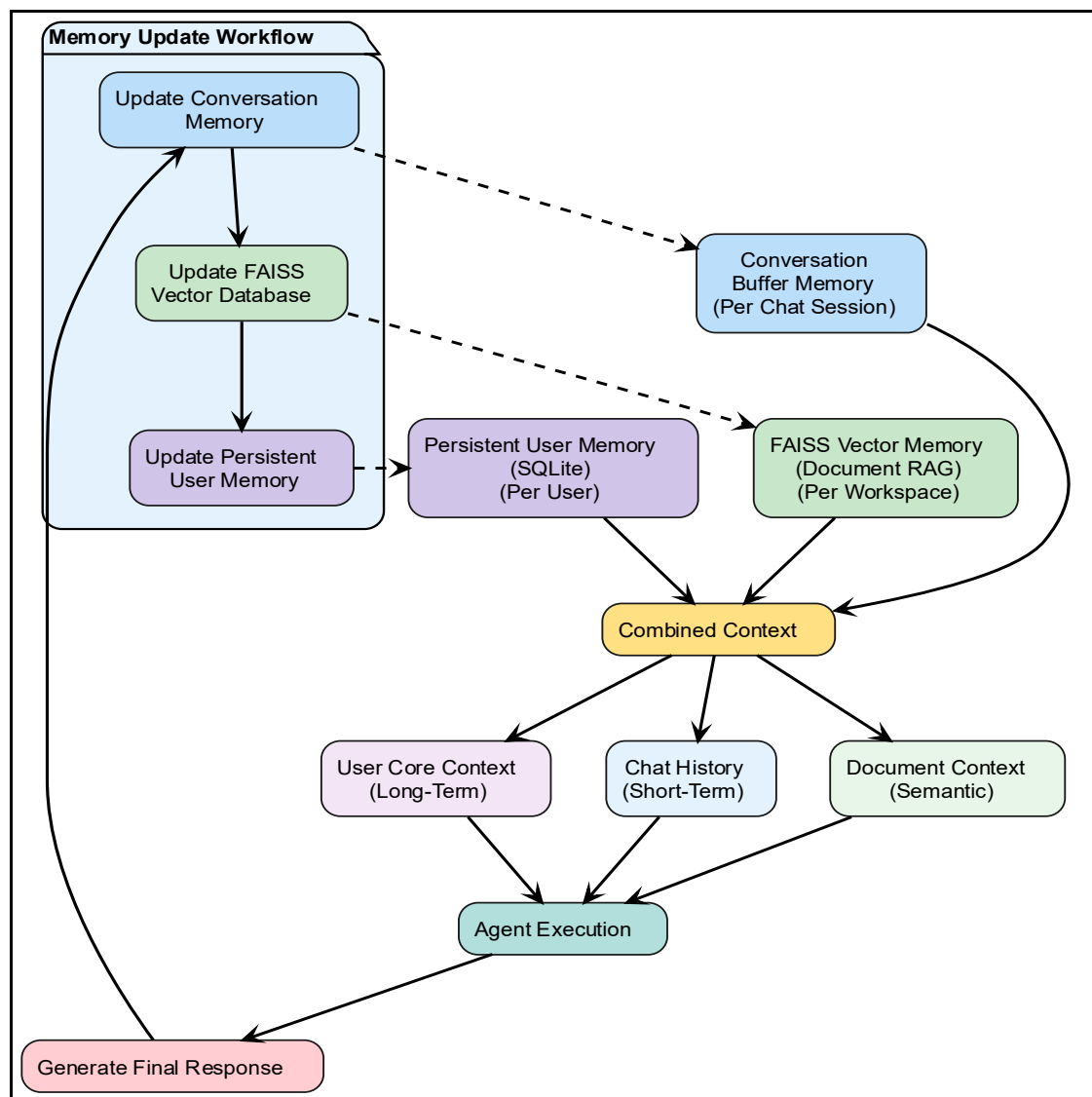


Fig 3.5: Memory Management Workflow

The orchestration framework implements a multi-layered memory management architecture for maintaining conversational continuity, semantic retrieval, contextual

reasoning, and personalized response generation across different workflows. Since standard Large Language Models are inherently stateless and cannot independently preserve interaction history, the framework integrates dedicated memory systems for storing and retrieving contextual information during runtime execution.

The memory architecture combines short-term conversational memory, semantic vector retrieval systems, and persistent user-specific storage mechanisms for improving workflow continuity and intelligent response generation. These memory systems operate collaboratively during query execution and dynamically provide contextual information to the orchestration engine and specialized agents [1].

The framework primarily uses three memory subsystems:

- Conversation Buffer Memory
- FAISS Vector Memory
- Persistent User Memory

These memory layers collectively improve contextual awareness, semantic retrieval accuracy, document understanding within the orchestration framework.

3.6.1 Conversation Buffer Memory

Conversation Buffer Memory is responsible for maintaining short-term conversational context during active user interactions within the orchestration framework. Since Large Language Models do not inherently preserve previous interactions, the framework integrates a dedicated conversational memory subsystem for storing and retrieving recent interaction history during runtime execution.

The framework maintains separate conversational memory instances for each individual chat workspace through an isolated memory registry architecture. This design prevents contextual interference between multiple sessions and ensures that every workspace preserves its own independent conversational state and execution history.

During query execution, the memory subsystem stores recent user inputs and generated system responses sequentially within the active session context. To improve runtime efficiency and reduce unnecessary storage overhead, large conversational entries are automatically truncated while retaining the most contextually important information required for subsequent reasoning operations. The stored conversational context is dynamically injected into the orchestration workflow before agent execution. This enables the framework to preserve contextual continuity across multi-turn

conversations and improves the ability of specialized agents to understand ongoing discussions, contextual references, and sequential interactions.

Conversation Buffer Memory plays an important role in maintaining natural conversational flow, contextual awareness, interactive dialogue continuity, and session-level reasoning throughout the orchestration process. The subsystem significantly improves the framework's ability to handle follow-up interactions and context-dependent conversational workflows without requiring repetitive user input during the same execution session [1].

3.6.2 FAISS Vector Memory

FAISS Vector Memory is responsible for semantic document retrieval and contextual information processing within the orchestration framework. The framework uses FAISS (Facebook AI Similarity Search) as a high-performance vector database for storing and retrieving semantic embeddings generated from uploaded documents and contextual content.

The FAISS memory architecture enables Retrieval-Augmented Generation (RAG) by allowing the orchestration system to retrieve contextually relevant document information before Large Language Model execution. This mechanism improves factual grounding, semantic understanding, and document-aware response generation during conversational workflows. The framework creates separate vector databases for individual workspaces and stores them independently within dedicated storage directories. This isolated vector-storage architecture ensures that semantic retrieval operations remain workspace-specific and prevents contextual interference between different chat sessions or uploaded document collections.

When users upload documents such as PDFs, DOCX files, spreadsheets, presentations, or text files, the orchestration engine first extracts textual content from the uploaded files. The extracted content is then divided into smaller semantic chunks using recursive text-splitting mechanisms for efficient embedding generation and retrieval operations.

Each chunk is converted into a dense vector representation using transformer-based embedding models. These semantic embeddings capture contextual relationships between document segments and are stored inside the FAISS vector database for similarity-search operations. During query execution, the user query is also transformed into a semantic embedding vector. The FAISS retriever performs similarity comparison

between the query embedding and stored document embeddings to identify contextually relevant document sections. The system retrieves the highest-matching semantic chunks and injects them into the orchestration workflow before Large Language Model execution.

The framework additionally supports both append mode and overwrite mode during vector database operations. Append mode allows newly uploaded documents to be added into an existing vector database without removing previously stored embeddings, while overwrite mode replaces the existing vector store with newly generated semantic content.

To maintain compatibility across different operating environments and workspace structures, the framework also implements safe path-handling mechanisms for vector database storage and retrieval operations.

FAISS Vector Memory significantly improves semantic retrieval accuracy, contextual document understanding, long-context interaction handling, Retrieval-Augmented Generation workflows, and factual response generation within the orchestration framework [6].

3.6.3 Persistent User Memory

Persistent User Memory acts as the long-term contextual memory layer of the orchestration framework. Unlike session-based conversational memory systems, this subsystem preserves user-specific contextual information across multiple sessions and execution cycles.

The framework stores persistent contextual information inside a dedicated SQLite-based memory table that maintains structured user-related data for future retrieval and personalization workflows. This memory architecture allows the system to preserve important contextual details such as user preferences, technical domains, development stacks, professional roles, workflow requirements, and interaction-specific personalization information.

The persistent memory subsystem operates using structured key-value contextual storage mechanisms. User-defined contextual facts are stored as semantic profile information and are dynamically retrieved during future interactions to improve adaptive response generation and personalized reasoning. The persistent memory architecture significantly improves long-term personalization, contextual continuity across sessions, adaptive conversational behaviour, intelligent recommendation

generation, and user-specific workflow customization within the orchestration framework [6].

3.7 Database Design

The orchestration framework uses a SQLite relational database for storing user information, conversational history, uploaded document metadata, persistent contextual memory, and workflow-related records. The database layer acts as the persistent storage backbone of the system and supports multiple orchestration operations including authentication, workspace management, memory retrieval, analytics tracking, and document processing workflows.

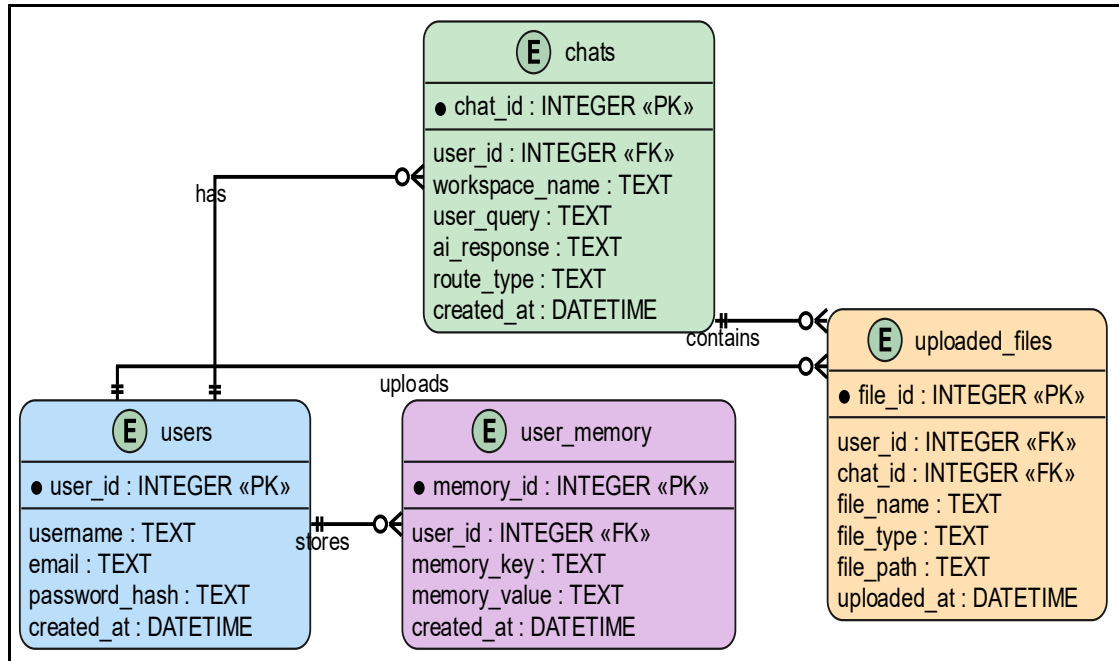


Fig 3.6: Database Schema Diagram (ER Diagram)

The database architecture was designed to maintain efficient data organization and seamless interaction between different orchestration components. Separate tables are used for managing user accounts, chat sessions, uploaded files, and long-term contextual memory records. This modular table structure improves data management, contextual retrieval, scalability, and workflow execution efficiency within the orchestration framework. The Users Table stores account-related information required for authentication and session management. This includes user registration details, login credentials, and account-specific information required during runtime execution.

The Chats Table maintains conversational history generated during user interactions with the orchestration framework. This table stores user queries, generated responses, execution metadata, routing information, timestamps, and workflow-related

contextual records. The stored conversational history supports workspace persistence, analytics generation, and contextual interaction tracking.

The User Memory Table is responsible for maintaining persistent user-specific contextual information across multiple sessions. This table stores long-term contextual memory records such as user preferences, technical domains, workflow preferences, and personalization-related information used during adaptive response generation.

The Uploaded Files Table stores metadata related to uploaded documents processed by the orchestration framework. This includes document names, file types, workspace associations, storage references, upload timestamps, and document-processing records used during Retrieval-Augmented Generation workflows. The relational database architecture significantly improves persistent storage management, contextual continuity, document tracking, user personalization, and orchestration workflow stability within the system.

SYSTEM IMPLEMENTATION

The backend of the Agent Orchestration Framework was implemented using Flask and Python. It acts as the core processing layer of the system and manages communication between the frontend interface, orchestration engine, agents, memory systems, databases, and external tools.

The backend handles user authentication, query processing, session management, workflow execution, file handling, database interaction, and response generation during runtime execution. Every user request is first received by Flask API routes and then forwarded to the orchestration engine for further processing. The backend architecture was developed using a modular implementation approach where different functionalities are separated into dedicated modules. The orchestration module manages workflow routing and execution pipelines, the agent module contains specialized agent definitions, the memory module handles contextual storage and retrieval, while the tools module manages integration with external APIs and utilities.

The Flask backend also supports asynchronous communication between the frontend dashboard and server-side orchestration workflows using AJAX requests. This allows the system to dynamically process queries and display AI-generated responses without requiring complete page reloads.

4.1 Backend Implementation

The backend of the Agent Orchestration Framework was developed using the Flask web framework in Python [3]. The backend serves as the central processing layer of the system and manages communication between the frontend interface, orchestration engine, database layer, memory systems, external tools, and AI agents.

The backend architecture follows a modular implementation approach in which different functionalities are separated into independent execution modules. This modular structure improves maintainability, scalability, debugging efficiency, and workflow organization within the framework. The Flask backend handles user requests received from the dashboard interface and forwards them to the orchestration engine for pipeline execution. During runtime execution, the backend processes user queries, uploaded documents, routing decisions, contextual memory retrieval, and AI-generated responses before returning the final output to the frontend interface.

The backend implementation also manages session handling and database connectivity for maintaining user workspaces, conversational history, uploaded files, and persistent contextual memory. Communication between the frontend and backend is performed asynchronously using AJAX requests, allowing dynamic response rendering without requiring full page reloads.

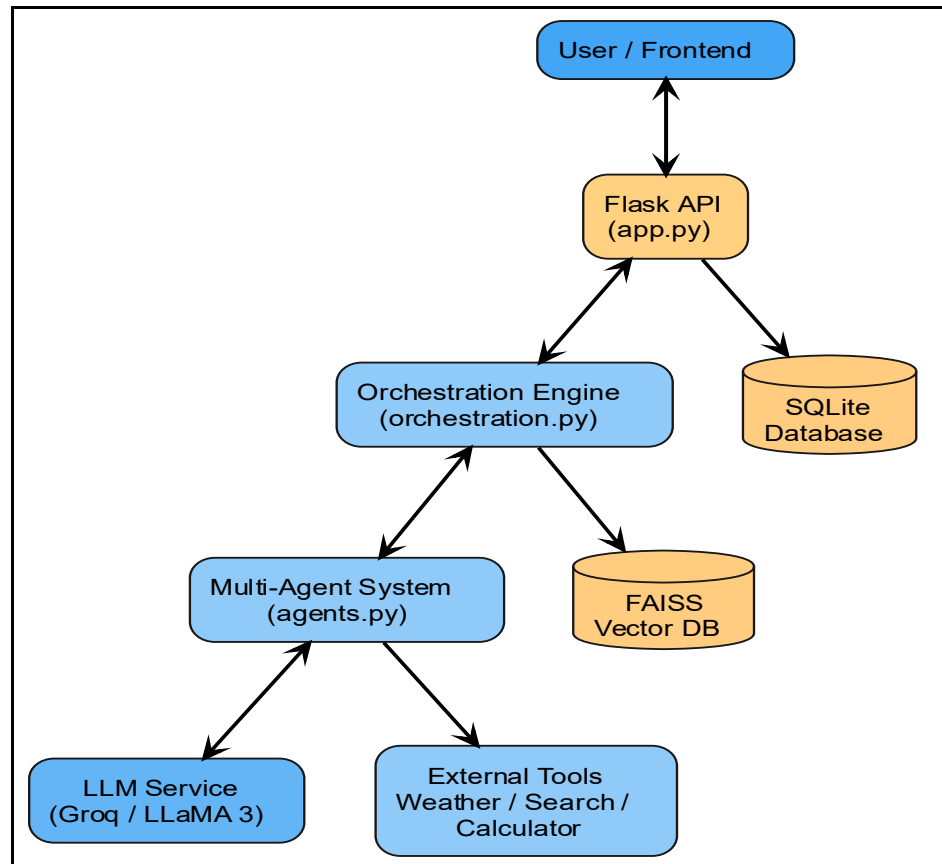


Fig 4.1: Backend Request-Response Workflow

The orchestration framework internally uses separate backend modules for workflow execution, specialized agent definitions, memory management, tool integration, and document processing operations. This separation of responsibilities improves execution flexibility and simplifies system maintenance during future development and enhancement phases.

4.2 API Endpoints and Session Management

The orchestration framework uses multiple Flask API endpoints for handling frontend communication, user authentication, query execution, workspace management, analytics retrieval, and document-processing operations. These API endpoints act as communication bridges between the frontend dashboard and backend orchestration engine during runtime execution. The primary execution endpoint receives user queries

from the chat dashboard and forwards them to the orchestration engine for workflow processing. Additional endpoints are used for operations such as workspace creation, workspace renaming, chat-history retrieval, analytics loading, message pinning, and persistent memory management.

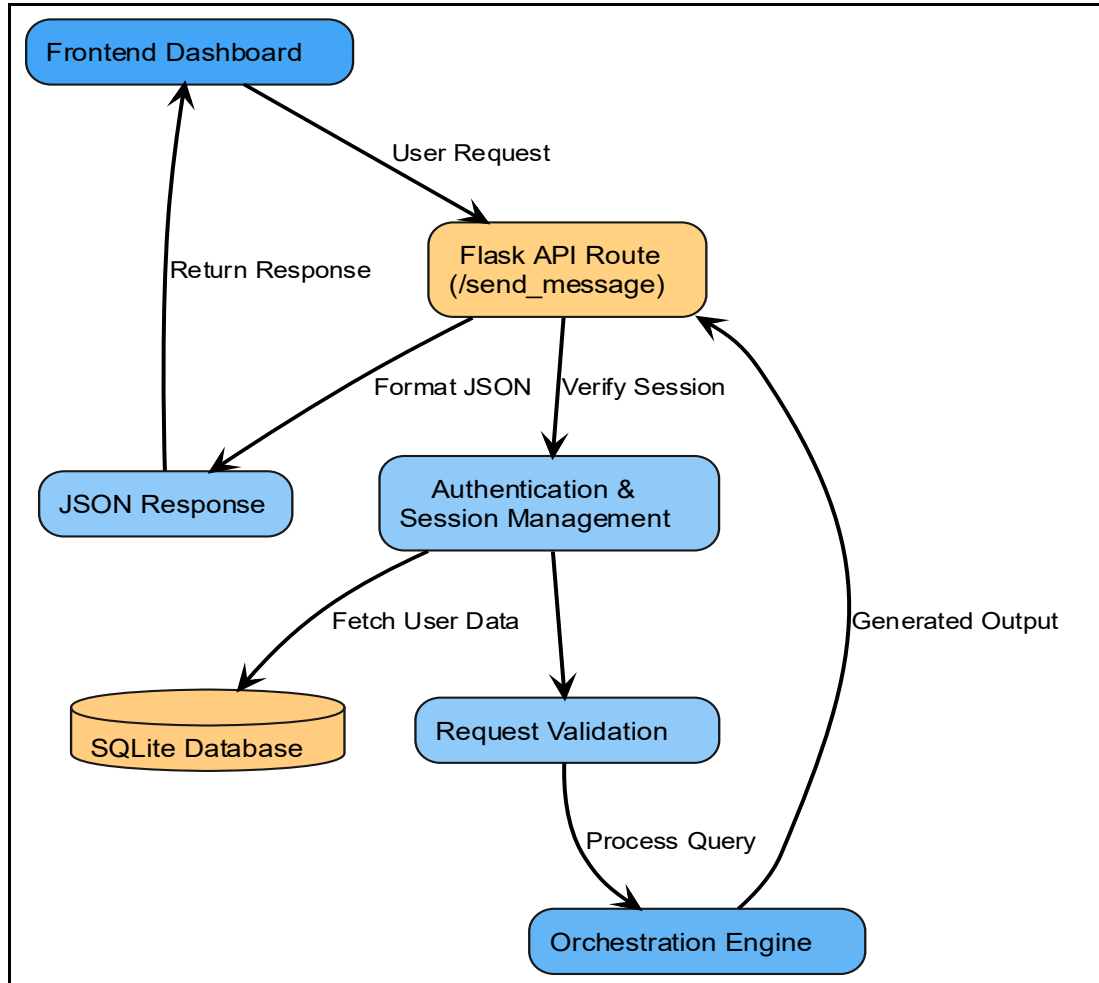


Fig 4.2: API Communication Workflow

The backend implementation supports asynchronous frontend communication through AJAX-based API requests. This allows the frontend dashboard to dynamically send user messages and receive AI-generated responses without requiring complete page refresh operations. The asynchronous communication architecture improves interaction smoothness and enhances user experience during conversational workflows.

Session management within the framework is implemented using Flask session handling mechanisms. User authentication status is preserved throughout active sessions to maintain secure access control across protected dashboard routes and orchestration operations. The framework additionally uses secure password hashing mechanisms during authentication workflows for protecting user credentials and

preventing direct password exposure inside the database layer. Session-based authentication enables users to maintain persistent access to workspaces, conversational history, uploaded documents, and contextual memory during active interactions.

Database connections are managed efficiently through backend connection-handling mechanisms to reduce unnecessary connection overhead and improve execution stability during multiple concurrent operations.

4.3 Tool Integration Implementation

The orchestration framework integrates multiple external tools for performing real-time operations and enhancing the reasoning capability of the AI system. These tools allow the framework to retrieve live information, solve mathematical expressions, perform web searches, process unit conversions, and generate contextual responses beyond the limitations of standalone language models.

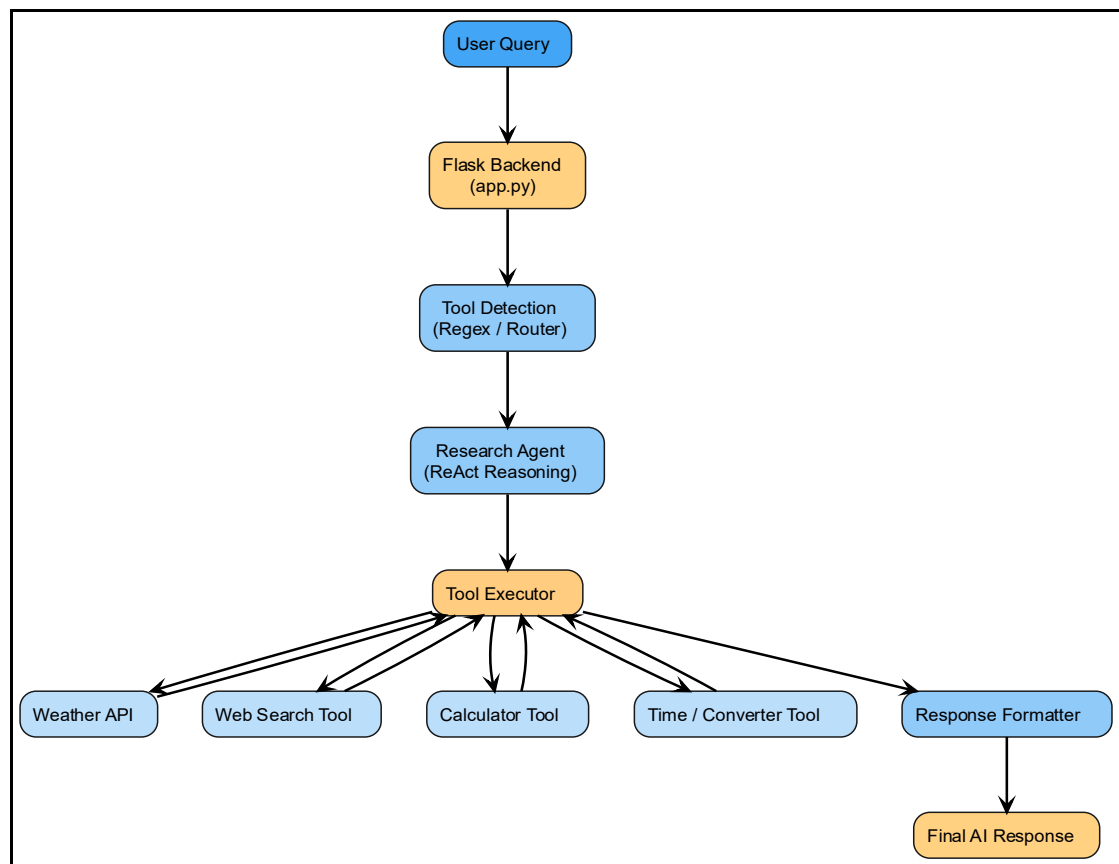


Fig 4.3: Tool Execution Workflow

The tool integration architecture is connected with the TOOL execution pipeline and Research Agent workflows. During execution, the framework dynamically selects the appropriate tool based on query intent, routing classification, and contextual analysis performed by the orchestration engine. The integrated tools improve response

accuracy, real-time information retrieval, computational capability, and external reasoning support within the orchestration framework [1].

1. Calculator Tool

The Calculator Tool was implemented using the SymPy mathematical computation library for solving arithmetic operations, algebraic equations, calculus expressions, and symbolic mathematical problems. The framework preprocesses mathematical expressions before execution to improve compatibility with Python-based symbolic computation syntax.

The calculator supports:

- arithmetic operations,
- algebraic equation solving,
- differentiation,
- integration,
- symbolic simplification,
- expression evaluation.

The orchestration engine automatically detects mathematical expressions within user queries and forwards them to the calculator execution module for processing[2].

2. Weather Tool

The Weather Tool was integrated using the Open-Weather-Map API for retrieving live weather information based on user-provided city names. The framework extract location entities from user queries and sends API requests to the weather service during runtime execution.

The weather subsystem retrieves:

- temperature,
- humidity,
- weather conditions,
- wind speed,
- and atmospheric information.

The generated weather information is formatted into structured responses before being returned to the user through the orchestration workflow [9].

3. Web Search Tool

The Web Search Tool was implemented using DuckDuckGo search integration for retrieving real-time web information during analytical and research-oriented

workflows. This tool enables the orchestration framework to access external information sources beyond the pretrained knowledge limitations of the language model. The Research Agent dynamically activates the search subsystem during:

- decision-making workflows,
- research queries,
- current-events analysis,
- shopping workflows,
- and technical troubleshooting operations.

Retrieved search results are summarized and formatted into context-aware responses before final output generation [1].

4. Unit Converter Tool

The Unit Converter Tool performs measurement conversion operations using regex-based value extraction and predefined unit-conversion logic. The framework automatically identifies numerical values and associated measurement units from user queries before performing conversion operations.

The converter subsystem supports multiple conversion categories including:

- length conversion,
- temperature conversion,
- weight conversion,
- metric-unit transformations.

The generated conversion result is dynamically formatted and injected into the orchestration response workflow [1].

5. System Time Tool

The System Time Tool provides real-time time-zone and location-based time retrieval functionality within the orchestration framework. The subsystem uses location mappings and time-zone-offset calculations for generating accurate local time responses for different geographical regions.

The framework extracts country or city names from user queries and retrieves corresponding time-zone information before generating the final response. If external time-zone information is unavailable, the framework uses fallback server-side time generation mechanisms. The System Time Tool improves real-time contextual interaction capability and supports globally distributed user interactions [1].

4.4 File Processing and Document Handling

The orchestration framework supports multi-format document processing for enabling Retrieval-Augmented Generation workflows and document-based conversational interactions. The file-processing subsystem allows users to upload documents directly through the dashboard interface and interact with the uploaded content using natural language queries.

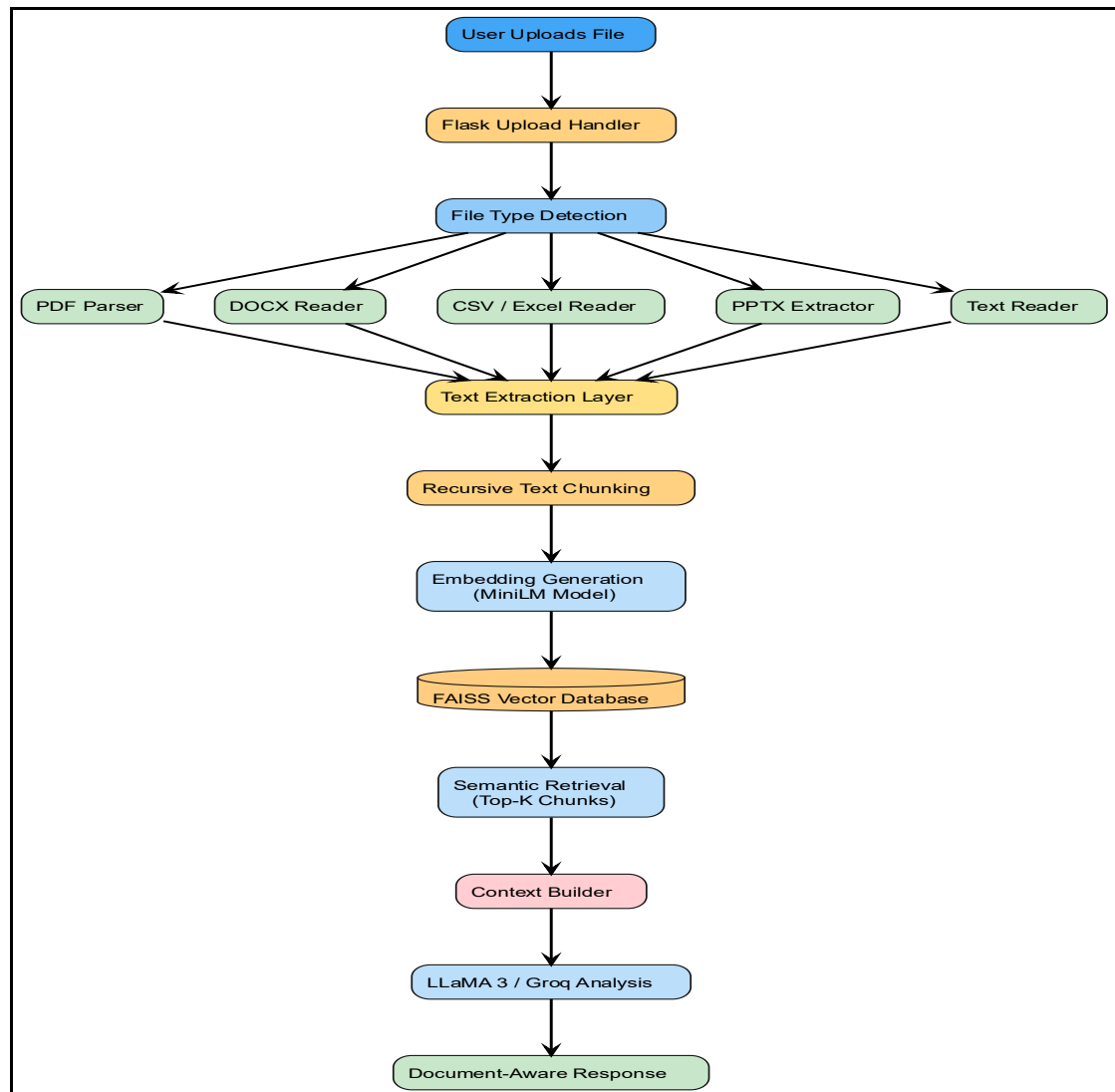


Fig 4.4: File Processing Pipeline

When a document is uploaded, the backend processing engine first detects the file type and selects the appropriate extraction mechanism for processing the document content. Different libraries and processing modules are used for handling different file formats within the orchestration workflow.

PDF documents are processed using text-extraction mechanisms that retrieve textual information from individual pages. Word documents are processed through

paragraph-based extraction methods, while spreadsheet files are converted into structured textual representations before semantic processing. PowerPoint presentations are analysed slide-by-slide for extracting textual content, headings, and presentation data. Plain text files are directly decoded and injected into the processing pipeline for semantic analysis.

After extraction, the document content is divided into smaller semantic chunks using recursive text-splitting mechanisms. These chunks are then converted into dense vector embeddings using transformer-based embedding models before being stored inside the FAISS vector database. During document-based query execution, the framework performs semantic similarity search between the user query and stored document embeddings. The retrieved contextual chunks are injected into the orchestration workflow before Large Language Model execution for generating context-aware responses.

The document-processing architecture significantly improves semantic retrieval capability, contextual document understanding, long-context interaction handling, and factual response generation within the orchestration framework [5].

4.5 Frontend Implementation

The frontend of the Agent Orchestration Framework was developed using HTML5, CSS3, and JavaScript to provide an interactive and responsive user interface for conversational AI interaction. The frontend acts as the communication layer between users and the backend orchestration system during runtime execution [3].

The interface was designed with a dashboard-oriented architecture to simplify user interaction with orchestration workflows, uploaded documents, analytics visualization, and conversational sessions. The frontend communicates dynamically with the Flask backend using asynchronous AJAX requests, enabling real-time interaction without requiring complete page refresh operations.

The chat interface allows users to submit queries, upload documents, receive AI-generated responses, and interact with multiple orchestration workflows through a unified conversational environment. The frontend additionally supports workspace-based interaction management, allowing users to organize conversations into separate contextual environments. Each workspace maintains independent conversational history, uploaded documents, and contextual memory associations during execution workflows [2].

The analytics interface provides visual monitoring capabilities for displaying workflow activity, response statistics, and system-usage trends using graphical representations and dashboard components. These visualization features improve monitoring capability and help users analyse orchestration-system behaviour more effectively. The frontend implementation significantly improves interaction smoothness, usability, accessibility, and overall user experience within the orchestration framework.

4.5.1 Landing Page Interface

The landing page serves as the primary entry point of the orchestration framework and provides navigation access to authentication workflows, project introduction, and conversational dashboard interaction. The interface was designed using responsive layout components and modern UI styling techniques for improving accessibility and visual presentation.

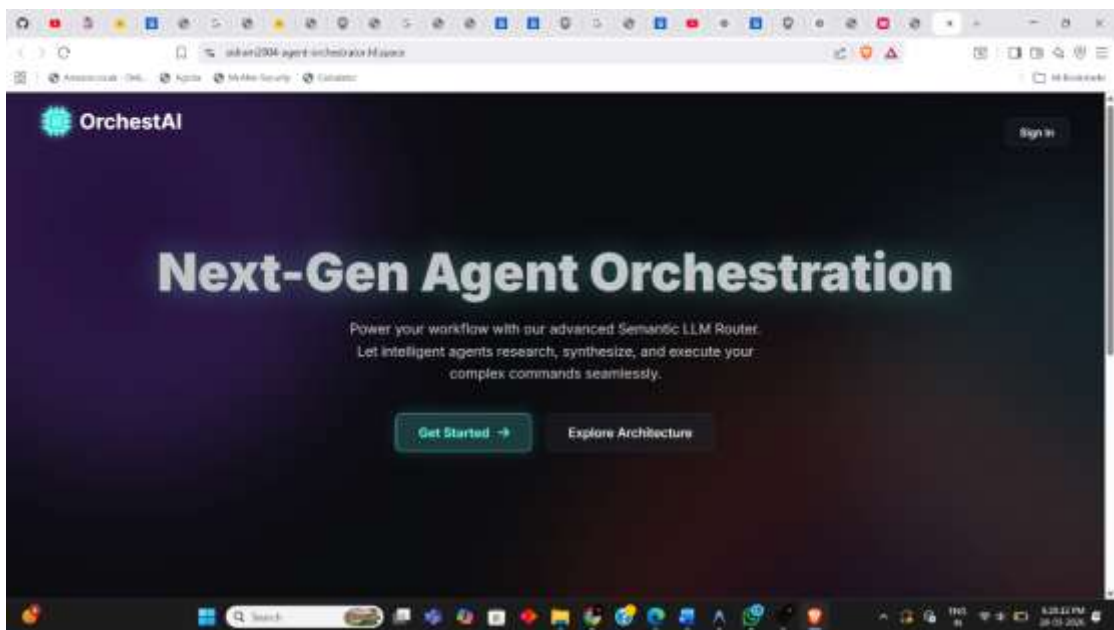


Fig 4.5: Landing Page Screenshot

4.5.2 User Registration and Login System

The authentication interface allows users to create accounts and securely access the orchestration dashboard using session-based authentication mechanisms. Registration and login forms were implemented using structured frontend validation and secure backend communication workflows. The interface was designed to provide simple navigation, responsive form handling, and protected user access during orchestration interaction.

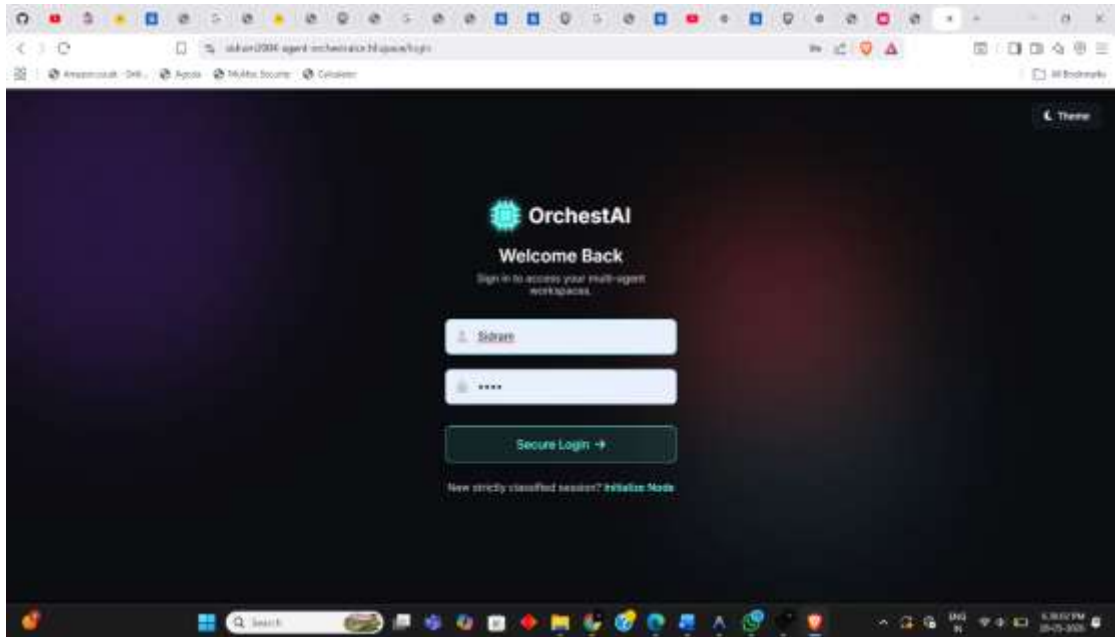


Fig 4.6: Registration and Login Interface

4.5.3 Main Chat Dashboard

The main chat dashboard acts as the central conversational environment of the orchestration framework. The interface supports conversational interaction, workspace navigation. The interface additionally displays routing indicators representing orchestration pipelines such as SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows during runtime execution.

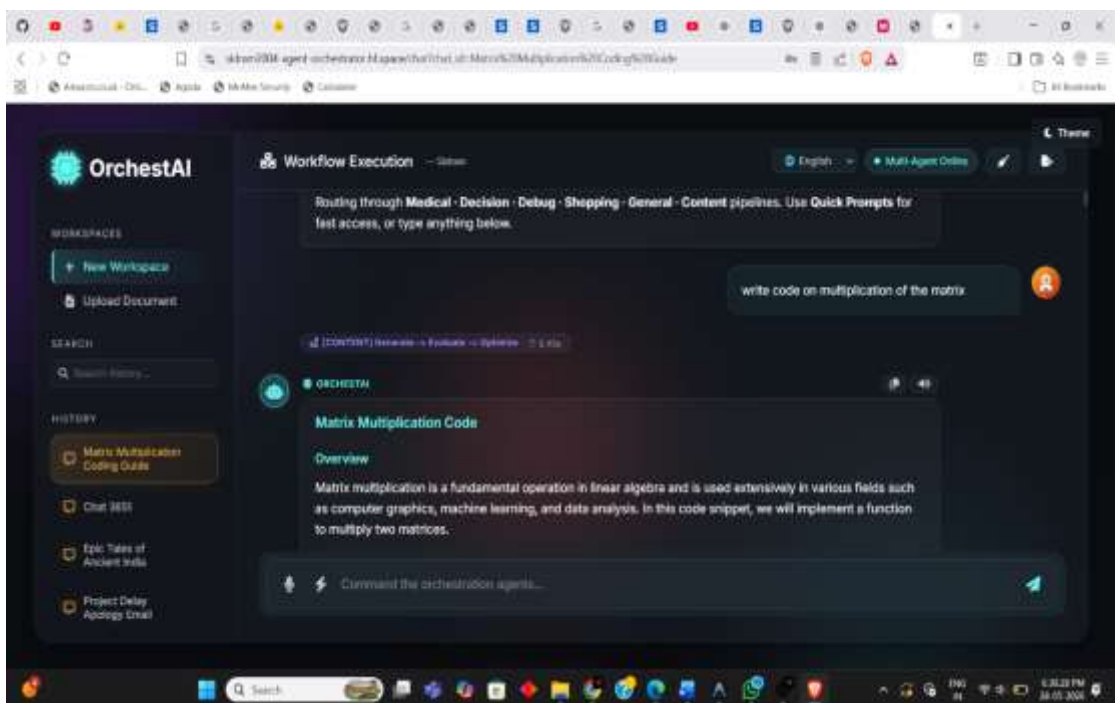


Figure 4.7: Main Chat Dashboard Screenshot

4.5.4 File Upload and Document Interaction

The frontend supports document-upload functionality for enabling Retrieval-Augmented Generation workflows during conversational interaction. Users can upload PDF, DOCX, CSV, PPTX, and text files directly through the dashboard interface for semantic document interaction. Uploaded documents are forwarded to backend processing workflows and integrated into semantic retrieval operations during document-aware conversational execution.

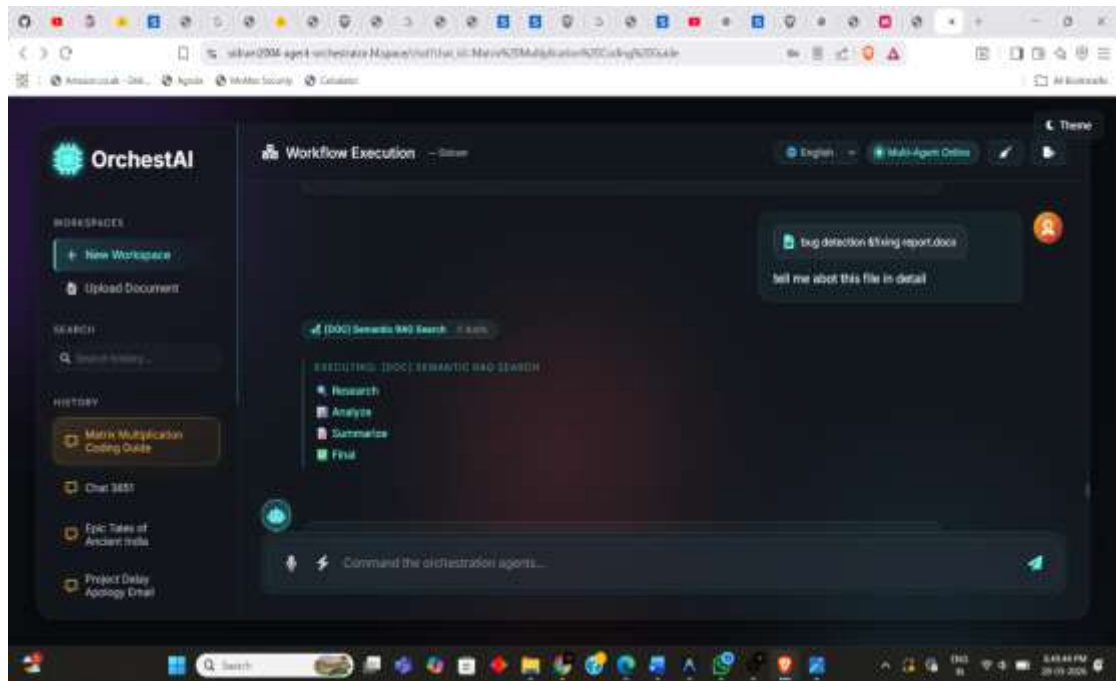


Figure 4.8: File Upload and RAG Interaction

4.6 Analytics Dashboard

The Analytics Dashboard was developed for monitoring system activity, workflow execution statistics, and user interaction trends within the Agent Orchestration Framework. The dashboard provides graphical visualization of orchestration performance and helps users analyse how different pipelines and agents are utilized during runtime execution.

The analytics interface was implemented using dynamic frontend components connected with backend API endpoints for retrieving real-time statistical data from the database layer. The dashboard continuously updates execution-related information and presents it through interactive visual elements and structured statistical panels.

The analytics subsystem displays information related to: total user interactions, total chat sessions, workflow execution distribution, average response times, message

activity trends, recent orchestration operations. Graphical charts and statistical cards are used for improving data readability and monitoring efficiency within the frontend interface. The dashboard also provides visual insight into how different orchestration pipelines such as SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows are utilized during system operation.

The backend analytics engine retrieves execution statistics from stored conversational history and processing logs within the SQLite database. This information is processed and dynamically rendered inside the frontend dashboard for generating real-time analytical summaries and usage reports.

The Analytics Dashboard improves system transparency, execution monitoring, workflow analysis, and operational visibility within the orchestration framework. It also assists in evaluating system performance, identifying usage patterns, and understanding orchestration behaviour during runtime execution.

4.7 Deployment and Execution Environment

The deployment architecture of the Agent Orchestration Framework was designed to support stable execution, modular scalability, and cloud-based accessibility for real-time conversational AI operations. The complete system was deployed using containerized infrastructure and integrated execution environments for ensuring consistent runtime behaviour across different platforms.

The framework was developed and executed using the Python programming environment with Flask acting as the primary backend execution server. The deployment configuration supports orchestration workflows, multi-agent execution, external tool integration, document processing, memory retrieval, and frontend interaction within a unified runtime architecture.

Containerization was implemented using Docker technology for simplifying dependency management and ensuring environment consistency during deployment. The Docker-based architecture packages the backend application, orchestration engine, frontend resources, dependencies, and execution libraries into a single isolated runtime container. This approach reduces configuration-related issues and improves portability across different systems and hosting environments. The framework was deployed on Hugging Face Spaces for cloud-based execution and remote accessibility. The deployment environment allows users to access the orchestration framework through a

browser-based interface without requiring local installation of dependencies or execution packages. The hosted deployment architecture also simplifies demonstration, testing, and project presentation workflows [7].

The execution environment includes integrated support for:

- Flask backend services,
- SQLite database operations,
- FAISS vector retrieval,
- memory management systems,
- document-processing libraries,
- and external API communication.

Environment variables were used for securely managing sensitive configuration parameters such as API keys, authentication tokens, and external service credentials. This approach improves deployment security and prevents direct exposure of confidential configuration information within the source code.

The deployment architecture additionally supports runtime interaction between frontend interfaces, orchestration pipelines, AI agents, database systems, memory layers, and external tools during live execution workflows. This integrated deployment model ensures smooth coordination between all major subsystems of the orchestration framework.

The deployment and execution environment significantly improves portability, execution consistency, scalability, maintainability, and cloud accessibility within the Agent Orchestration Framework.

RESULTS AND EVALUATION

This chapter presents the testing outcomes, performance evaluation, execution results, and overall system behaviour of the Agent Orchestration Framework. The evaluation process was conducted to analyse the efficiency of query routing, multi-agent execution, document processing, memory retrieval, external tool integration, and contextual response generation within different orchestration workflows.

The framework was tested using multiple categories of conversational and analytical queries to evaluate execution stability, routing accuracy, response quality, and workflow coordination. Different orchestration pipelines including SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows were analysed under various runtime scenarios for measuring overall system performance and operational reliability.

The evaluation process additionally focused on functional correctness, response latency, semantic retrieval capability, contextual memory handling, and adaptive conversational behaviour during user interaction. The obtained results demonstrate the effectiveness of the orchestration architecture in managing specialized workflows, collaborative agent execution, and Retrieval-Augmented Generation operations within a unified conversational AI environment.

5.1 Functional Testing

Functional testing was conducted to verify whether the Agent Orchestration Framework correctly performs all major system operations and orchestration workflows during runtime execution. The testing process focused on validating query routing, multi-agent execution, document processing, memory retrieval, external tool integration, and response generation across different execution pipelines.

Different categories of user queries were executed within the orchestration environment to evaluate workflow behaviour and system stability under multiple operational conditions. The testing process also verified authentication workflows, workspace management operations, uploaded-document handling, persistent memory retrieval, analytics functionality, and conversational interaction continuity.

The orchestration framework successfully processed conversational requests belonging to SIMPLE, TOOL, CONTENT, COMPLEX, and DOC pipelines. The routing engine correctly classified user queries into their appropriate execution

workflows and dynamically activated the required specialized agents during runtime execution. The TOOL pipeline successfully executed external operations such as weather retrieval, mathematical computation, web search, time-zone handling, and unit conversion. The CONTENT workflow generated structured responses including essays, explanations, and formatted textual outputs using collaborative content-generation agents.

The COMPLEX pipeline demonstrated stable multi-agent coordination during advanced analytical workflows involving debugging, decision analysis, shopping recommendations, and generalized reasoning tasks. The DOC pipeline successfully processed uploaded documents through semantic retrieval and Retrieval-Augmented Generation mechanisms using FAISS vector search.

Functional testing additionally verified memory continuity and contextual personalization features within the orchestration framework. Persistent user memory and conversational memory systems successfully preserved contextual information across interactions and improved adaptive response generation during follow-up conversations.

Test Case ID	Functionality Tested	Input/Action	Expected Result	Actual Result	Status
TC-01	User Registration	Create new account	Account created	Account created successfully	Pass
TC-02	User Login	Login with valid credentials	Redirect to dashboard	Login successful	Pass
TC-03	SIMPLE Pipeline	What is AI?	SIMPLE routing	Correct response generated	Pass
TC-04	TOOL Pipeline	Weather in Mumbai	Weather tool executes	Weather response generated	Pass
TC-05	CONTENT Pipeline	Write internship email	Content generation	Structured content generated	Pass
TC-06	COMPLEX Pipeline	Debug Python code	Complex reasoning	Debug response generated	Pass
TC-07	DOC Pipeline	Upload PDF and ask question	Semantic retrieval	Document-aware response	Pass
TC-08	FAISS Retrieval	Search uploaded document	Relevant chunks retrieved	Correct retrieval performed	Pass
TC-09	Persistent Memory	Save user preference	Recall stored memory	Personalized response generated	Pass
TC-10	Analytics Dashboard	Open analytics	Charts displayed	Dashboard loaded successfully	Pass

Table 5.1: Functional Test Cases

The testing process confirmed that the orchestration framework maintained stable execution behaviour, correct workflow routing, reliable tool interaction, and context-aware response generation across all major operational scenarios.

5.2 Performance Evaluation

Performance evaluation was conducted to analyse the execution efficiency, response-generation speed, routing accuracy, and workflow stability of the Agent Orchestration

Framework under different operational conditions. The evaluation process measured how effectively the orchestration engine handled user queries across multiple execution pipelines during runtime interaction.

Pipeline Type	Description	Avg Response Time	Min Time	Max Time	Throughput	Status
SIMPLE	Basic conversational queries	1.82 sec	1.15	2.87	32.97 req/min	Pass
TOOL	API and tool execution	2.35 sec	1.45	3.91	25.53 req/min	Pass
CONTENT	Content generation workflow	3.21 sec	2.01	4.78	18.69 req/min	Pass
COMPLEX	Multi-agent reasoning tasks	4.87 sec	3.12	8.98	12.32 req/min	Pass
DOC	RAG document retrieval	10.76 sec	5.45	15.67	15.96 req/min	Pass

Table 5.2: Pipeline Response Time Analysis

Different categories of conversational, analytical, and document-based queries were executed repeatedly to observe processing behaviour and pipeline performance consistency. The evaluation process focused on response latency, orchestration overhead, semantic retrieval efficiency, external tool execution time, and multi-agent coordination behaviour during workflow execution.

The SIMPLE pipeline demonstrated the fastest execution performance because queries were directly forwarded to the language model without activating additional orchestration layers or external reasoning workflows. This pipeline produced low-latency responses suitable for lightweight conversational interaction and direct informational requests.

The TOOL pipeline required moderate execution time due to communication with external services and real-time information retrieval operations. Response latency varied depending on external API availability, network communication, and tool-processing complexity during runtime execution.

The CONTENT and COMPLEX pipelines required comparatively higher processing time because multiple specialized agents participated in reasoning, evaluation, optimization, refinement, and validation workflows before final response generation. Despite additional orchestration overhead, the framework maintained stable execution behaviour and structured output generation during advanced analytical tasks.

The DOC pipeline additionally performed semantic retrieval operations using FAISS vector search before language-model execution. Uploaded documents were

processed through embedding retrieval and contextual chunk extraction mechanism before generating document-aware responses. Although semantic retrieval introduced additional processing overhead, the framework maintained efficient contextual response generation during Retrieval-Augmented Generation workflows. The routing engine demonstrated high classification accuracy during orchestration execution and successfully reduced unnecessary computational overhead by dynamically selecting optimized workflows based on query complexity and contextual requirements.

The evaluation results indicate that the orchestration framework maintained stable performance, efficient workflow coordination, reliable routing behaviour, and context-aware response generation across different execution scenarios.

5.3 System Results

The Agent Orchestration Framework successfully demonstrated intelligent workflow execution, multi-agent coordination, Retrieval-Augmented Generation, contextual memory retrieval, and real-time tool integration within a unified conversational AI environment. The obtained results confirmed that the orchestration architecture was capable of handling diverse categories of user queries through dynamically selected execution pipelines.

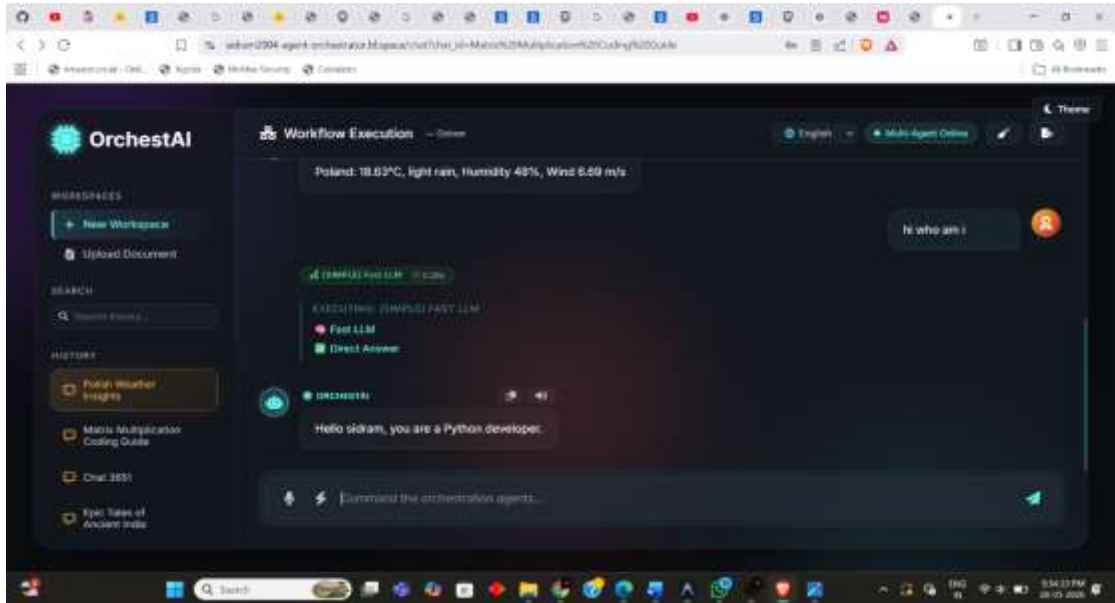


Fig 5.1: SIMPLE Pipeline Output

The SIMPLE pipeline generated fast and stable responses for lightweight conversational interactions and direct informational requests. Queries related to greetings, definitions, and basic factual questions were processed efficiently with minimal orchestration overhead and low response latency.

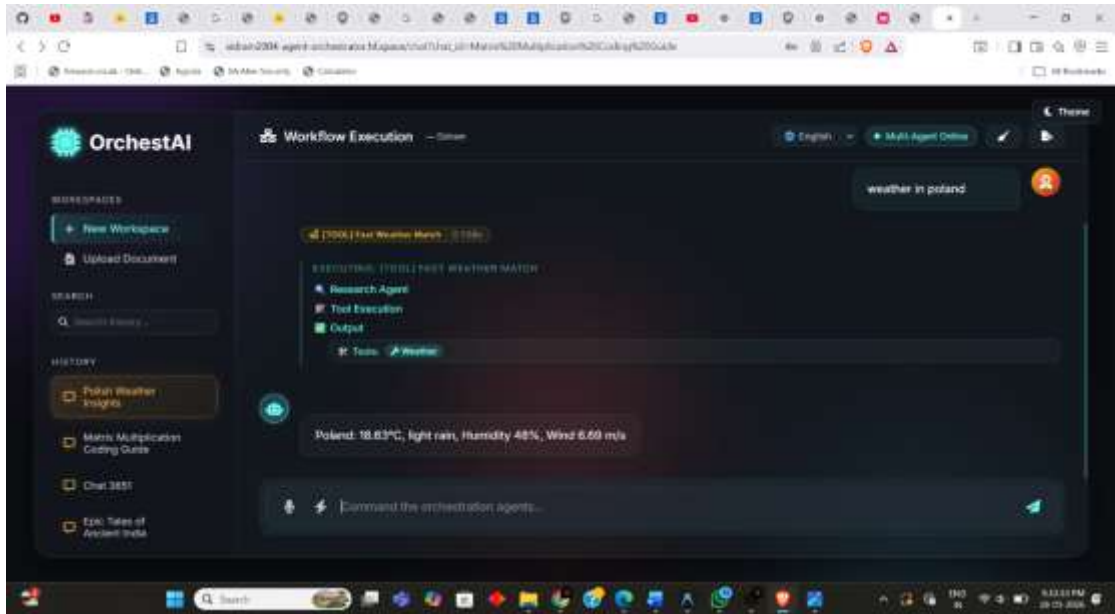


Fig 5.2: TOOL Pipeline Output

The TOOL pipeline successfully executed real-time operations through external service integration. The framework accurately performed mathematical computation, weather retrieval, web search, time-zone tracking, and unit-conversion operations during runtime execution. Tool-generated outputs were correctly formatted and integrated into conversational responses before final delivery.

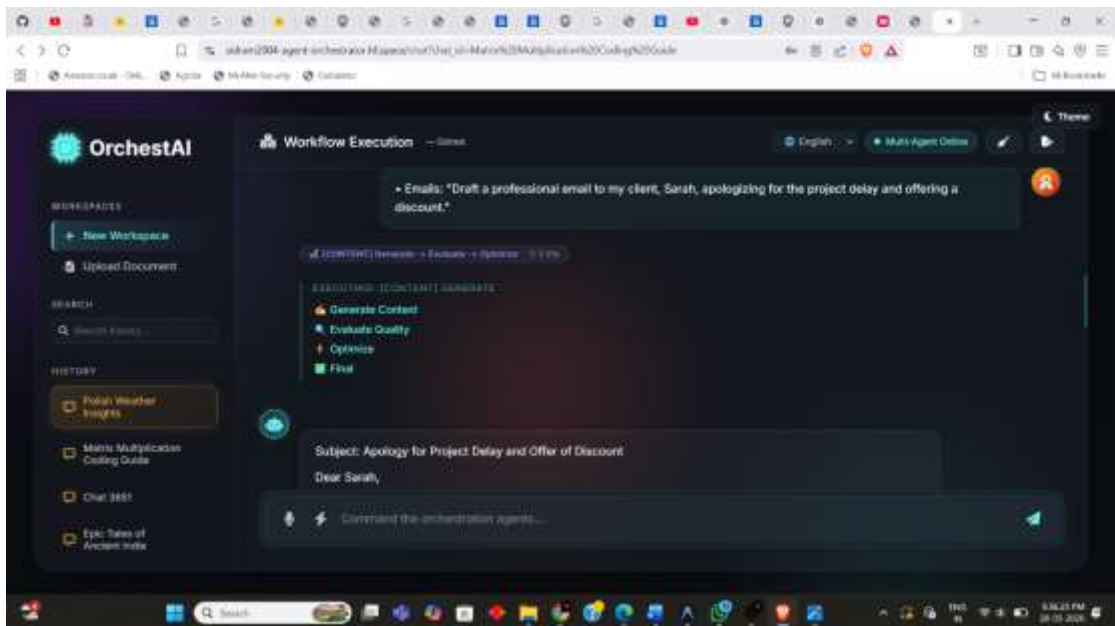


Fig 5.3: CONTENT Pipeline Output

The CONTENT pipeline demonstrated effective collaborative content-generation capability using multiple specialized agents. The framework generated structured outputs such as essays, explanations, formatted documents, and descriptive content

through iterative evaluation and optimization workflows. The generated responses maintain readability, structural consistency, and contextual relevance during execution.

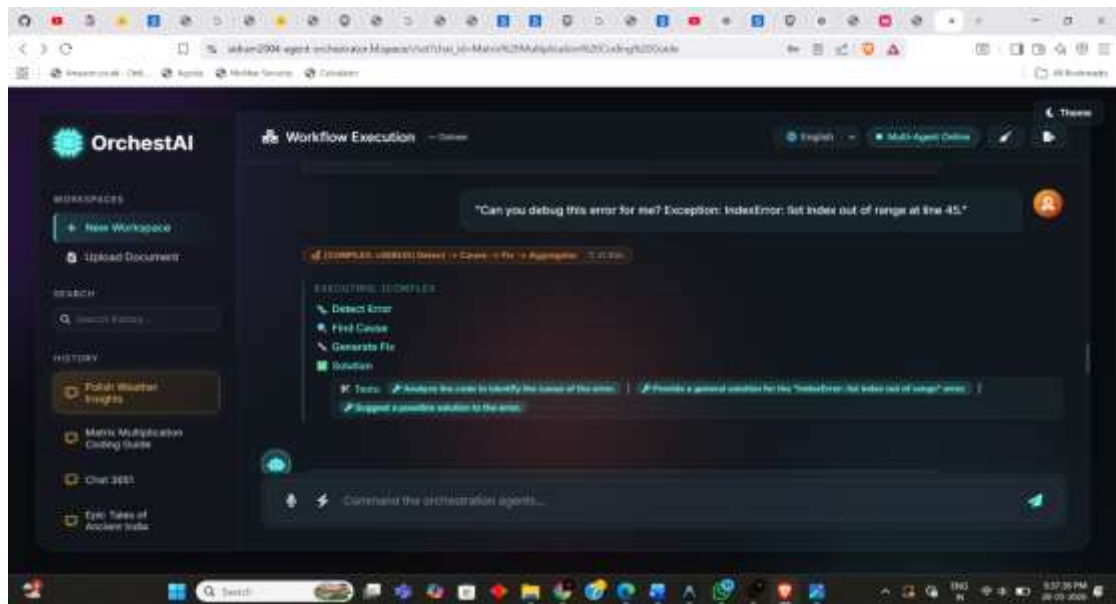


Fig 5.4: COMPLEX Pipeline Output

The COMPLEX pipeline demonstrated advanced reasoning capability through coordinated multi-agent execution workflows. Specialized agents successfully handled debugging operations, decision-analysis tasks, shopping recommendations, medical reasoning workflows, and generalized analytical interactions using contextual orchestration strategies.

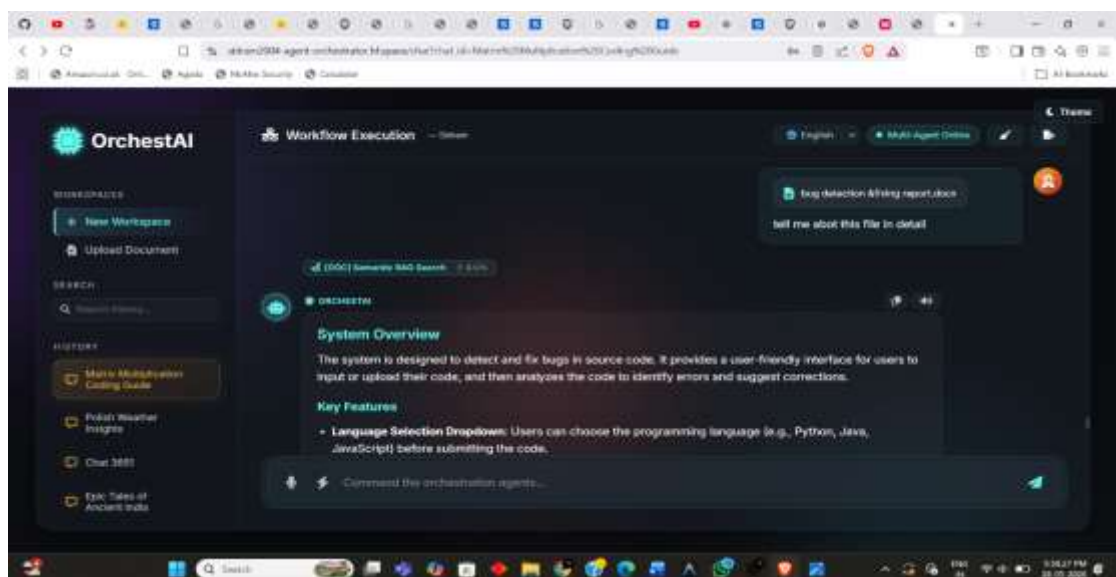


Fig 5.5: DOC Pipeline Output

The DOC pipeline successfully implemented Retrieval-Augmented Generation through semantic document retrieval and contextual response generation. Uploaded documents

were processed through embedding generation and FAISS vector retrieval mechanisms before contextual chunks were injected into orchestration workflows. The framework generated accurate document-aware responses based on semantically retrieved information from uploaded files.

The system additionally demonstrated persistent memory capability by retrieving previously stored user information and dynamically injecting contextual personalization during future interactions. This improved conversational continuity and adaptive response generation across multiple sessions.

The system results confirm that the orchestration framework successfully integrated workflow routing, multi-agent coordination, semantic retrieval, memory management, and external tool execution within a stable conversational AI architecture.

5.4 Advantages of the System

The Agent Orchestration Framework provides several advantages compared to traditional single-model conversational AI systems. The integration of multiple specialized agents improved workflow distribution, contextual reasoning, response quality, and execution flexibility during runtime interaction.

The intelligent query-routing mechanism reduced unnecessary computational overhead by dynamically selecting optimized execution pipelines based on query complexity and contextual requirements. Lightweight queries were processed rapidly through simplified workflows, while complex analytical tasks activated specialized multi-agent orchestration mechanisms. The framework successfully integrated external tools for performing real-time operations such as weather retrieval, mathematical computation, web search, time-zone tracking, and unit conversion. This significantly improved real-time information access and reduced dependency on static pretrained model knowledge.

Retrieval-Augmented Generation improved document-aware conversational capability by enabling semantic interaction with uploaded files through FAISS vector retrieval mechanisms. The framework successfully generated contextually grounded responses using semantically retrieved document chunks during runtime execution. The persistent memory architecture improved conversational continuity and personalization by preserving user-specific contextual information across multiple sessions. This enhanced adaptive interaction capability and allowed the system to generate more context-aware responses during follow-up conversations.

The modular system architecture additionally improved maintainability, scalability, debugging efficiency, and future extensibility within the orchestration framework. Independent orchestration modules simplified workflow management and enabled easier integration of additional agents and external tools during future development phases. The framework also demonstrated stable frontend-backend communication, dynamic dashboard interaction, workspace-based conversation management, and efficient document-processing capability within a unified conversational AI environment.

5.5 Limitations of the System

Although the Agent Orchestration Framework demonstrated stable execution capability and advanced orchestration functionality, certain operational limitations were identified during system evaluation and runtime testing. The framework depends on external APIs for certain real-time operations such as weather retrieval and web-search functionality. External service delays, network instability, or API rate limitations may affect response-generation speed during high-frequency execution scenarios.

The FAISS vector-retrieval subsystem currently operates within a CPU-based execution environment. Large-scale document collections may increase semantic retrieval overhead and reduce processing efficiency during intensive Retrieval-Augmented Generation workflows. The framework currently uses SQLite as the primary database engine for lightweight storage management. While suitable for small-scale deployment and academic implementation, SQLite may not provide optimal scalability for distributed enterprise-level production environments involving high-concurrency execution workloads. Certain orchestration workflows involving multiple specialized agents may introduce additional processing latency because responses are generated only after all reasoning, evaluation, optimization, and refinement stages are completed.

The current implementation also does not support token-level streaming responses during runtime interaction. Users receive generated responses after complete workflow execution rather than progressively during generation. Despite these limitations, the framework maintained stable orchestration behaviour, contextual response generation, and workflow coordination across different operational scenarios during evaluation.

5.6 Future Enhancements

Several improvements can be integrated into the Agent Orchestration Framework to enhance scalability, execution intelligence, workflow flexibility, and real-time interaction capability during future development phases. Future implementation may include graph-based orchestration architectures such as Lang-Graph for enabling stateful execution workflows, conditional branching, cyclical reasoning, and advanced agent coordination. This would improve workflow control and allow more dynamic orchestration strategies during complex analytical operations.

Parallel agent execution mechanisms can additionally be integrated for reducing execution latency during multi-agent workflows. Instead of sequential execution, multiple specialized agents could process independent subtasks simultaneously before combining outputs into a final response. The current SQLite database architecture may be replaced with scalable database systems such as PostgreSQL or MongoDB for supporting large-scale deployment environments and concurrent multi-user interaction workflows. Distributed caching systems such as Redis may also be integrated for improving retrieval speed and execution efficiency. Future versions of the framework may additionally token-level streaming responses, voice-based conversational interaction, advanced semantic chunking, multilingual conversational support, real-time collaborative workspaces, administrative monitoring systems.

The Retrieval-Augmented Generation subsystem can further be improved through hybrid vector-search architectures and advanced embedding models for increasing semantic retrieval accuracy during document-based interaction workflows. Cloud-native deployment architectures and container orchestration systems may also be integrated for improving scalability, fault tolerance, and distributed execution capability during production deployment.

CONCLUSION

The Agent Orchestration Framework with LangChain successfully demonstrated the development of an intelligent multi-agent conversational AI system capable of dynamic query routing, contextual reasoning, Retrieval-Augmented Generation, persistent memory management, and external tool integration within a unified architecture.

The framework integrated sixteen specialized AI agents, five external utility tools, and five orchestration pipelines including SIMPLE, TOOL, CONTENT, COMPLEX, and DOC workflows for handling diverse conversational and analytical tasks. The implementation of FAISS-based semantic retrieval and persistent contextual memory significantly improved document-aware interaction, response relevance, and personalized user experience. The developed system effectively addressed several limitations of traditional standalone Large Language Models by introducing workflow specialization, collaborative agent execution, semantic document retrieval, real-time tool interaction, and long-term contextual memory capabilities.

The virtual internship and project development process provided valuable practical experience in conversational AI engineering, Flask backend development, AI orchestration, vector databases, Retrieval-Augmented Generation, frontend integration, API communication, and cloud-based deployment technologies.

The project demonstrates the growing importance and future potential of multi-agent AI systems in building scalable, adaptive, context-aware, and intelligent conversational platforms for real-world applications.

REFERENCES

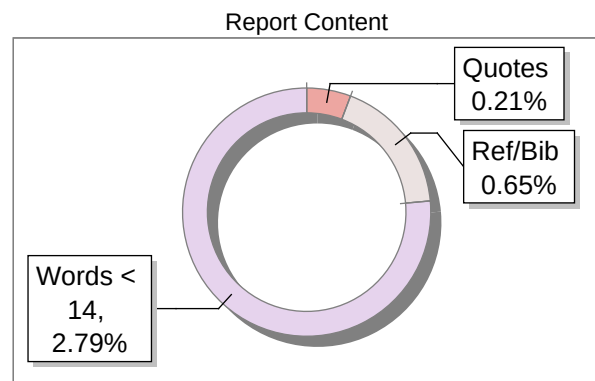
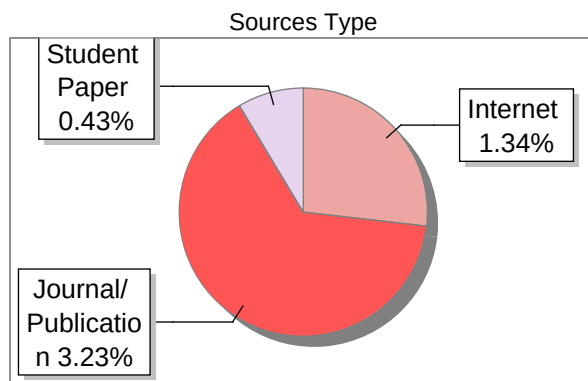
- [1] H. Chase, “LangChain Documentation.” [Online]. Available: <https://python.langchain.com/docs/>
- [2] Meta AI, “LLaMA 3 Large Language Model.” [Online]. Available: <https://ai.meta.com/llama/>
- [3] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed., O'Reilly Media, 2018.
- [4] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021.
- [5] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.
- [6] Faiss AI, “FAISS: Facebook AI Similarity Search.” [Online]. Available: <https://faiss.ai/>
- [7] Docker Inc., “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>
- [8] Hugging Face, “Hugging Face Spaces Documentation.” [Online]. Available: <https://huggingface.co/docs/hub/spaces>
- [9] OpenWeatherMap, “Weather API Documentation.” [Online]. Available: <https://openweathermap.org/api>
- [10] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O'Reilly Media, 2022.

Submission Information

Author Name	Patil Sidram Bajrang
Title	final merged
Paper/Submission ID	5801961
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-06-02
Total Pages, Total Words	92, 19741
Document type	Project Report

Result Information

Similarity **5 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

5

SIMILARITY %

70

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	www.ir.juit.ac.in8080	<1	Publication
2	ijmtst.com	<1	Publication
3	batch.libretexts.org	<1	Publication
4	Analytical ultracentrifugation of colloids by Planken-2010	<1	Publication
5	REPOSITORY - Submitted to Exam section VTU on 2026-02-05 15-33 4442828	<1	Student Paper
6	ijert.org	<1	Internet Data
7	REPOSITORY - Submitted to Exam section VTU on 2026-02-11 16-27 4422204	<1	Student Paper
8	The GRADE taxonomy for supporting decision making asset selection in , by Papatheocharous, Ef- 2018	<1	Publication
9	Effects of proteinprotein interaction in ultrafiltration based fractionation by Carlo-2005	<1	Publication
10	REPOSITORY - Submitted to Exam section VTU on 2026-02-10 10-58 4374119	<1	Student Paper
11	Thesis Submitted to Shodhganga, shodhganga.inflibnet.ac.in	<1	Publication
12	ipsc.fudutsinma.edu.ng	<1	Publication

13	Thesis Submitted to Shodhganga Repository	<1	Publication
14	Agent-based guitar performance simulation by Leandr-2008	<1	Publication
15	arxiv.org	<1	Publication
16	dbatu.ac.in	<1	Internet Data
17	joirem.com	<1	Publication
18	pdfcookie.com	<1	Internet Data
19	www.freepatentsonline.com	<1	Internet Data
20	ijarcce.com	<1	Publication
21	www.tandfonline.com	<1	Internet Data
22	confcats-siteplex.s3.us-east-1.amazonaws.com	<1	Publication
23	iiit.ac.in	<1	Internet Data
24	www.europarl.europa.eu	<1	Publication
25	bioresources.cnr.ncsu.edu	<1	Internet Data
26	ijsred.com	<1	Publication
27	library.oapen.org	<1	Publication
28	www.paperdigest.org	<1	Internet Data
29	apps.dtic.mil	<1	Publication
30	encord.com	<1	Internet Data
31	pmc.ncbi.nlm.nih.gov	<1	Internet Data

32	Semantic information and knowledge integration through argumentative reasoning t by Janjua-2013	<1	Publication
33	AN APPLICATION OF THE ISOIEC 25010 STANDARD IN THE QUALITY-IN-USE ASSESSMENT OF by Hussain-2015	<1	Publication
34	dochero.tips	<1	Internet Data
35	Fast Revelation of the Motif Mode for a Yeast Protein Interaction Network Throug by Lee-2010	<1	Publication
36	nvpi.vn	<1	Internet Data
37	REPOSITORY - Submitted to Exam section VTU on 2024-08-01 15-39 1021652	<1	Student Paper
38	Thesis submitted to dspace.mit.edu	<1	Publication
39	Thesis Submitted to Shodhganga Repository	<1	Publication
40	www.advamed.org	<1	Publication
41	www.jetir.org	<1	Publication
42	ACM Press the 1st international conference- Macao, China (2007.12., by Bjrgvinsson, Trygg- 2007	<1	Publication
43	ACM Press the 2008 ACM symposium- Fortaleza, Ceara, Brazil (2008.0, by Borsje, Jethro Lev- 2008	<1	Publication
44	arxiv.org	<1	Internet Data
45	arxiv.org	<1	Internet Data
46	arxiv.org	<1	Publication
47	category-theory.mitpress.mit.edu	<1	Internet Data

48	CFD-population balance Monte Carlo simulation and numerical optimization for fla by Xu-2016	<1	Publication
49	digitalscholarship.unlv.edu	<1	Publication
50	dochero.tips	<1	Internet Data
51	docplayer.net	<1	Internet Data
52	encord.com	<1	Internet Data
53	files.eric.ed.gov	<1	Publication
54	ijircce.com	<1	Publication
55	Optimization of drug-drug interaction alert rules in a pediatric hospitals elec by Simpao-2014	<1	Publication
56	pmc.ncbi.nlm.nih.gov	<1	Internet Data
57	proceedings-of-deim.github.io	<1	Publication
58	REPOSITORY - Submitted to Exam section VTU on 2026-02-11 10-57 4393870	<1	Student Paper
59	Scale Space View of Curve Estimation by Proba-2000	<1	Publication
60	Thesis submitted to shodhganga - shodhganga.inflibnet.ac.in	<1	Publication
61	Thesis Submitted to Shodhganga Repository	<1	Publication
62	Using a web-based categorization approach to generate thematic metada, by Huang, Chien-Chung- 2004	<1	Publication
63	www.leewayhertz.com	<1	Internet Data
64	www.medrxiv.org	<1	Publication
65	www.medrxiv.org	<1	Publication

66	www.neiu.edu	<1	Internet Data
67	www.sap.com	<1	Internet Data
68	www.scitepress.org	<1	Publication
69	www.scribd.com	<1	Internet Data
70	IEEE 2014 IEEE 22nd Annual International Symposium on Field-Programmable by	<1	Publication